

normet

Normalisation, Decomposition and Counterfactual Modelling for Environmental Time Series

User Manual

the window of the normet R package

Version 1.0.0

An R package, and a desktop window on it, for removing the weather from air-quality records and asking what an intervention changed.

October 21, 2026

Contents

1 Introduction **3**
- 1.1 What is normet? . . . 3
- 1.2 The idea in one equation . . . 3
- 1.3 Key features . . . 4
- 1.4 How this manual is organised . . . 4

2 Installation **5**
- 2.1 R and the normet package . . . 5
- 2.2 macOS . . . 5
- 2.3 Windows and Linux . . . 5
- 2.4 From the repository . . . 6

3 Methodology **7**
- 3.1 Preparing the data . . . 7
  - 3.1.1 The split . . . 7
- 3.2 Training the model . . . 8
  - 3.2.1 Model statistics . . . 8
- 3.3 Meteorological normalisation . . . 8
- 3.4 Emission decomposition . . . 9
- 3.5 Meteorological decomposition . . . 9
- 3.6 Rolling normalisation . . . 10
- 3.7 Multi-scale decomposition . . . 10
- 3.8 Partial dependence . . . 10
- 3.9 Synthetic control . . . 11
  - 3.9.1 Estimators . . . 11
  - 3.9.2 Diagnostics . . . 11
  - 3.9.3 Inference . . . 12

4 Getting started **13**
- 4.1 The window before a table is open . . . 13
- 4.2 The examples . . . 14
- 4.3 A first analysis . . . 14

5 The interface **15**
- 5.1 The parameter panel . . . 15
- 5.2 Result tabs . . . 15
- 5.3 The action bar . . . 16
- 5.4 Run buttons . . . 16
- 5.5 Menus and shortcuts . . . 17
- 5.6 The log and the Assistant . . . 17
- 5.7 Appearance and language . . . 18

6 Training a model **19**
6.1 The table and its columns . . . 19
6.2 Settings . . . 19
6.3 Reading the Model tab . . . 21

7 Normalising **22**

8 Decomposing **24**
8.1 By time scale: the emission decomposition . . . 24
8.2 By variable: the meteorological decomposition . . . 25

9 Rolling and multi-scale **26**
9.1 Rolling normalisation . . . 26
9.2 Multi-scale decomposition . . . 26

10 Partial dependence **28**

11 Synthetic control (SCM Studio) **29**
11.1 Panel data and columns . . . 29
11.2 Design . . . 29
11.3 Fitting . . . 30
11.4 Inference . . . 32

12 Getting data in **34**
12.1 Input format . . . 34
12.2 The clock: time zone . . . 34
12.3 Data Studio: UK air quality and meteorology . . . 34
12.4 Transport Studio: back-trajectory predictors . . . 35
12.5 Polar plot: where a pollutant comes from . . . 36

13 Exporting and reproducing **39**
13.1 Tables and reports . . . 39
13.2 The command log . . . 39
13.3 Figures for publication . . . 41
13.4 Saving and reopening a session . . . 43
13.5 A Methods paragraph . . . 43
13.6 Configurations and history . . . 43

14 The Assistant **45**
14.1 Full automatic . . . 45
14.2 Choosing a model . . . 45

15 Scripting: R and MCP **47**
15.1 The R package . . . 47
15.2 Rscript . . . 48
15.3 AI assistants over MCP . . . 48

A Defaults **49**

B Where normet keeps its files **50**

C Troubleshooting **51**

Chapter 1

Introduction

1.1 What is normet?

normet is an R package, with a desktop window, for the analysis of environmental time series, air-quality records above all. It answers two questions that a raw concentration record cannot answer by itself:

1. *How much of the change I see is the weather?* A month of stagnant, cold anticyclonic air raises concentrations as surely as a new source does. normet learns how the pollutant depends on the meteorology and the calendar, then averages the meteorology out, leaving the part of the record the weather does not explain. This is *meteorological normalisation*, or *de-weathering*.
2. *What would have happened without the intervention?* A low-emission zone, a lockdown or a plant closure changes one place at one date. normet builds a counterfactual for that place from other places the intervention did not touch — a *synthetic control* — and measures the difference after the date, with placebo tests and uncertainty bands to say whether the difference is larger than chance.

Everything the window does is a call into the normet R package, run by `Rscript` in an R process of its own; the same functions are scripted in R, or driven by an AI assistant over the Model Context Protocol (Chapter 15). The window writes down each call it makes, so an analysis done by clicking can be saved as an R script that repeats it (Section 13.2). The window is normet-py's, the same screen for screen; what it runs is R.

1.2 The idea in one equation

Let y_t be the concentration at time t, $\mathbf{m}_t$ the meteorology at that time (wind speed and direction, temperature, humidity, boundary-layer height, ...) and $\mathbf{c}_t$ the calendar (a trend counter, the day of the year, the weekday, the hour). normet fits a machine-learning model

$$y_t \approx f(\mathbf{m}_t, \mathbf{c}_t) \tag{1.1}$$

and then asks what f predicts at time t if the weather at t had been any other weather the site has seen. Averaging over many such draws,

$$\tilde{y}_t \;=\; \frac{1}{N}\sum_{k=1}^{N} f\big(\mathbf{m}^{(k)}, \mathbf{c}_t\big), \qquad \mathbf{m}^{(k)} \sim \text{the record's own meteorology}, \tag{1.2}$$

gives the *normalised* series $\tilde{y}_t$: the concentration expected at time t under typical weather. Because the calendar $\mathbf{c}_t$ is kept, a change in emissions — which the model can only express through the trend, season, week and hour — survives in $\tilde{y}_t$, while the day-to-day swings that the weather caused are averaged away. Chapter 3 develops this and the decompositions built on it.

1.3 Key features

- Training with LightGBM (a random hyperparameter search) or H2O AutoML (an algorithm search within a time budget), with the held-out fit reported as R^2, RMSE and nine further statistics.
- Monte-Carlo meteorological normalisation, with a choice of which variables are resampled, uncertainty quantiles, reference pools and conditional resampling; an adaptive version that stops when the series has converged.
- Emission decomposition (trend, seasonal, weekly, diurnal, short-term) and meteorological decomposition (one contribution per variable or named group, sequential or Shapley).
- Rolling (windowed) normalisation, and a multi-scale decomposition that separates the weather's influence by timescale.
- Partial-dependence curves of the trained model.
- Synthetic control on panel data: augmented SCM, classic Abadie SCM, difference-in-differences, matrix completion, robust SCM, ML-SCM and Bayesian SCM, with placebo-in-space and placebo-in-time tests, jackknife and bootstrap bands, and an all-units batch.
- Data Studio: UK air quality from six monitoring networks with reanalysis meteorology, merged into a model-ready table without an API key.
- Transport Studio: HYSPLIT back-trajectories turned into transport predictors and joined onto the record.
- A command log that writes every action as the call that repeats it; HTML reports with provenance; figures laid out for a journal's column.
- An Assistant (a language model working in the window) that proposes the target and predictors and reads the results with you.

1.4 How this manual is organised

Chapter 2 covers installation. Chapter 3 explains the methods in enough depth to interpret every number the program reports. Chapters 4–10 walk through a complete de-weathering analysis in the window on a real record, and Chapter 11 the synthetic-control workflow on a real panel. Chapter 12 covers getting data in, Chapter 13 exporting, reporting and reproducing a session, Chapter 14 the Assistant and Chapter 15 scripting. The appendices list the defaults, where normet keeps its files, and a troubleshooting guide.

The figures are screenshots of the real window, made by `gui/tools/walk_gui.py` on the record normet ships with: hourly NO_2 and ERA5 meteorology at London Marylebone Road (site MY1), January to August 2020, a window that spans the UK's first COVID-19 lockdown. The numbers quoted in the text are the ones that script printed. A run of your own should match them closely: LightGBM's search and every draw are seeded, though LightGBM's threads may move a last digit.

This manual is also published in Chinese (`normet_manual_zh.pdf`), chapter for chapter, with its screenshots of the window in Chinese (Section 5.7).

Chapter 2

Installation

The window runs the normet R package, so R and the package come first; then the window, as an application or from the repository.

2.1 R and the normet package

normet needs R 4.1 or newer. In R:

```
install.packages(c("remotes", "lightgbm"))
remotes::install_github("apai-sys/normet-r")
install.packages("h2o") # optional: the H2O AutoML backend (needs Java)
```

LightGBM is the default backend. H2O AutoML is optional and needs a Java runtime; the window starts its cluster when an H2O step runs and shuts it down when the window closes. The window finds `Rscript` on the `PATH` or where R installs it; **R → Locate Rscript...** points it at another, and **R → Check R environment** asks again which R, which normet and which backends there are. The status bar shows R's version, its tooltip normet's and the backends that load; a notice says what to install when something is missing.

2.2 macOS

Open `Normet-1.0.0-macos-arm64.dmg` and drag `Normet.app` onto the `Applications` folder. The application carries its own Python and Qt, not R: it runs the R installed on the machine. The first launch takes longer than the rest while it builds a font cache, which later launches reuse (it is kept in `~/.normet/matplotlib`); a splash screen shows meanwhile.

Note. The application is not signed with an Apple Developer ID. The first time, open it by right-clicking `Normet.app` and choosing *Open*, then confirming in the dialog; this is needed only once. Alternatively run `xattr -dr com.apple.quarantine /Applications/Normet.app` in Terminal.

2.3 Windows and Linux

On Windows run `normet-setup-1.0.0.exe`, an Inno Setup wizard that adds Start-menu and optional desktop shortcuts. The installer is unsigned, so SmartScreen shows *Windows protected your PC* on first run: choose *More info → Run anyway*. On Linux download `Normet-1.0.0-x86_64.AppImage`, mark it executable (`chmod +x`) and run it. On a minimal distribution Qt may need `libegl1 libgl1 libxkbcommon0 libdbus-1-3 libxcb-cursor0`.

The installers are built by the repository's `build-gui.yml` workflow (one per operating system; PyInstaller cannot cross-compile) and are attached to its runs under *Actions → Artifacts*. `gui/packaging/README.md` explains how to build one locally.

2.4 From the repository

The window is a Python package in the repository's `gui` folder. It needs Python 3.10 or newer:

```
python -m venv .venv && source .venv/bin/activate
pip install "git+https://github.com/apai-sys/normet-r#subdirectory=gui"
normet-r-gui # or: normet-r-gui mydata.csv
```

Its extra `geo` adds Shapely and pyshp, for source regions read from GeoJSON or Shapefile files in the Transport Studio.

Chapter 3

Methodology

This chapter explains what normet computes, so that the numbers it reports can be interpreted with confidence. You do not need it to run an analysis; you do need it to defend one.

3.1 Preparing the data

Before a model is trained the table goes through `nm_prepare_data`, in this order:

1. **A date column.** The table must have one datetime column called `date` (or an index, or a column named `datetime`, `time` or `timestamp`, which is renamed). Dates with time zones are converted to UTC and then made naive.
2. **Target and predictors.** The target is renamed `value`; the table is reduced to the date, the target and the predictors.
3. **Missing values.** Rows with a missing target are dropped. A missing numeric predictor is filled with that column's median; a missing categorical one with its most frequent value.
4. **Calendar variables.** Four columns are added: `date_unix` (seconds since 1970, the trend), `day_julian` (day of the year, the season), `weekday` (1 = Monday to 7 = Sunday, categorical) and `hour`. The model uses only those listed among its predictors; the window always lists all four.
5. **Train/test split.** A column `set` marks each row `training` or `testing` (Section 3.1.1).

3.1.1 The split

The held-out rows are what the model's fit is judged on. Four splits are offered:

`random` (default) A random 25 % of rows are held out. Every season, weekday and hour is represented in training.

`ts` The last 25 % of the record is held out. A trend variable must then be extrapolated past anything seen in training, which tree models cannot do; use it to test forecasting, not to train for normalisation.

`month_ts` Within every calendar month of every year, one contiguous block of 25 % is held out, at a random (seeded) position in the month.

`season_ts` The same within every meteorological season (DJF, MAM, JJA, SON; December counts with the following year's winter).

Caution. Under `random`, neighbouring hours fall on both sides of the split, so an autocorrelated record looks easier to predict than it is: the test R^2 is an upper bound. This matters most for predictors that are constant over several hours, such as back-trajectory features released every six hours, which a tree model can use as a time stamp. On the bundled MY1 $PM_{2.5}$ case a trajectory-only model scores about 0.84 under a random split and 0.46 under a blocked one. Judge such models with `month_ts`.

3.2 Training the model

normet trains gradient-boosted trees (by default) that map the predictors to the target. Two backends are offered:

LightGBM (the default) LightGBM with a random hyperparameter search of a set number of *trials*, each scored by cross-validation (50 trials in the library, 20 in the window).

H2O AutoML An AutoML search over the algorithms ticked (GBM alone by default; DRF, GLM, deep learning, XGBoost and stacked ensembles can be added) within a wall-clock *time budget* (`max_runtime_secs`, 60 s), on a local H2O cluster.

LightGBM's search is seeded, so the same script on the same data finds the same model. Because H2O's budget is wall-clock time, the same script can stop at a different candidate on a busier machine; set `max_models` in `model_config` for a search bounded by models instead.

3.2.1 Model statistics

`nm_modStats` predicts the training rows, the testing rows and all rows, and reports for each: n; FAC2 (the fraction of predictions within a factor of two of the observation); MB and MGE (mean bias and mean gross error); NMB and NMGE (the same, normalised by the sum of observations); RMSE; r and its significance; COE (coefficient of efficiency); IOA (index of agreement); and R^2. The held-out (set = testing) row is the one to judge the model by.

3.3 Meteorological normalisation

Equation (1.2) is computed by Monte Carlo. For each of N draws (`n_samples`, 300 by default):

1. The *resampled variables* of every row are replaced, all together, by those of a row drawn at random (with replacement) from the *resample pool* — by default the record itself. Whole rows are drawn, so the resampled variables keep their joint structure: a draw never pairs a winter temperature with a summer boundary layer.
2. Every other predictor — by default the four calendar variables — keeps its observed value.
3. The model predicts every row.

The normalised series is the mean of the N predictions at each timestamp; with `return_quantiles` the chosen quantiles of the N predictions are returned as well, as columns `q050`, `q950` and so on (the quantile $\times 1000$). These quantiles describe the spread of the prediction over weather, not the uncertainty of the model.

Which variables to resample. Resampling the meteorology and keeping the calendar is the usual de-weathering. Resampling a calendar variable too removes what it carries: resampling `day_julian` removes the seasonal cycle, resampling `date_unix` removes the trend. A predictor that is not meteorological — traffic counts, say — can be kept at its observed values by leaving it unticked.

How many samples. The Monte-Carlo error of each timestamp shrinks as $1/\sqrt{N}$. 300 is ample for a smooth daily or monthly mean; individual hours are noisier. `nm_normalise_auto` chooses N for you: it adds batches of 100 draws until, for three checks in a row, 95 % of the timestamps have a Monte-Carlo relative standard error below 3 %, or 5000 draws have been made (a warning says when the cap was hit).

Reference pools. `resample_df` draws the weather from another table — 2019's weather, for example, to ask what 2020 would have looked like under 2019's meteorology. `conditional_on` (of `nm_normalise_ext`) restricts the pool to rows meeting a condition — a value, a vector of values, or a function of the column, as `list(ws = function(v) v < 2)` draws from calm hours only. `resample_pools` draws some variables from a pool of their own: `list(transport = clean_hours[traj_cols])` draws the trajectory features from clean-air-mass hours while the local weather is still drawn from the whole record. Each pool has its own random stream, so adding or renaming a pool does not move the draws of the others.

Reproducibility. The draws are seeded (`seed = 7654321` by default), so the same model, data and settings give the same series to the last digit.

3.4 Emission decomposition

The emission decomposition splits the normalised series into the timescales the calendar variables carry, by *nested* normalisation. Five normalisations are run with the same seed, fixing one more calendar variable each time, in this fixed order:

Run	Held at observed values	Resampled	Gives
b	nothing	everything	base
u	`date_unix`	the rest	trend $= u - b$
j	... + `day_julian`	the rest	seasonal $= j - u$
w	... + `weekday`	the rest	weekly $= w - j$
h	... + `hour`	meteorology	diurnal $= h - w$

The last run, h, is the ordinary normalised series, reported as `emi_total`. The base run is split into its mean, `emi_base`, and its deviation from that mean, `emi_noise`; so

$$\text{emi_total} = \text{emi_base} + \text{date_unix} + \text{day_julian} + \text{weekday} + \text{hour} + \text{emi_noise}. \quad (3.1)$$

A calendar variable that is not among the model's predictors gets no column.

Caution. The order decides what each component can hold. The trend is computed while the day of the year, weekday and hour are still being averaged over, so it cannot carry a signal that recurs at the same date every year — only a genuine drift. The seasonal component is computed with each row's own trend value already fixed, so it is not a climatology: it stays specific to its year and can register a one-off event, such as a single year's holiday dip or a lockdown, despite its label. Look for a recurring calendar effect in `day_julian`, not `date_unix`.

3.5 Meteorological decomposition

The meteorological decomposition splits the part of the prediction the weather accounts for among the meteorological variables, or among named groups of them. The calendar is held at its observed values throughout. Write $v(S)$ for the normalised series with the variables in S held at their observed values and the rest resampled; $v(\emptyset)$ is `emi_total`, and $v(\text{all})$ is the model's prediction with nothing resampled. The difference $v(\text{all}) - v(\emptyset)$ is split in one of two ways:

Sequential Fix the variables one at a time and report each step's change: $\phi_i = v(\{1, \ldots, i\}) - v(\{1, \ldots, i-1\})$. This takes $k+1$ normalisations for k variables, but each contribution is conditional on those fixed before it, so the split depends on the order — an explicit `variable_order`, else the model's feature importance, which can reshuffle when the model is refitted.

Shapley Average each variable's marginal effect over every order it could be fixed in:

$$\phi_i = \sum_{S \not\ni i} \frac{|S|!\,(k - |S| - 1)!}{k!} \left[v(S \cup \{i\}) - v(S) \right]. \tag{3.2}$$

No order is privileged. The exact values need all 2^k coalitions, allowed up to $k = 10$; beyond that, group the variables or pass `n_permutations` for an estimate from random orders (drawn in antithetic pairs).

Either way the contributions add up exactly to $v(\text{all}) - v(\emptyset)$, and every normalisation uses the same seed, so the differences between them are paired (common random numbers) rather than the difference of two noisy estimates. The result also carries `met_total` = observed − `emi_total`, its mean `met_base`, and `met_noise` = `met_total` − `met_base` − $\sum_i \phi_i$, the part of the observation the model does not reproduce.

Groups. `groups = list(local = met_cols, transport = traj_cols)` gives one column per group; a group is fixed and resampled as a unit, so its column includes the interactions among its members. Every meteorological predictor must be in exactly one group. With groups the default attribution is Shapley; without them it is sequential.

3.6 Rolling normalisation

`nm_rolling` normalises the record window by window: windows of `window_days` days (14 by default), started every `rolling_every` days (7), each normalised with its weather drawn *from that window only*. A short window therefore removes only the weather that varies within it — synoptic variability for a fortnight — and keeps the slower meteorological variation that a full-record normalisation would remove. The result has one column per window, counted from 1: `window_01`, `window_02`, ... (padded to three digits past 99 windows, so the names sort in the windows' order); `window_columns(rolled)` lists them. The window shows their mean, with the spread across overlapping windows.

3.7 Multi-scale decomposition

Write Y_W for the mean of the rolling-normalised series at window width W, and Y_∞ for the ordinary normalised series, whose pool is the whole record. As W grows the window's pool approaches the record's, and $Y_W \to Y_\infty$. Differences between widths isolate the weather influence specific to each band of timescales, much as wavelet detail coefficients do:

$$D_{\text{fast}} = Y_{\text{fast}} - Y_{\text{meso}} \qquad \text{synoptic, sub-monthly} \tag{3.3}$$

$$D_{\text{meso}} = Y_{\text{meso}} - Y_{\text{slow}} \qquad \text{intra-seasonal} \tag{3.4}$$

$$D_{\text{slow}} = Y_{\text{slow}} - Y_\infty \qquad \text{what even the slow window's pool leaves} \tag{3.5}$$

The widths default to 14, 90 and 365 days. Each scale's windows are stepped every quarter of the width, and a scale with fewer than two windows on the record is skipped, with a note saying so.

3.8 Partial dependence

A partial-dependence curve shows how the model's prediction responds to one predictor with the others averaged over the data: for each of `grid_points` values spanning the 1st to 99th percentile of the variable, every training row's value of it is set to that value and the predictions are

averaged. `pdp_mean` is that average and `pdp_std` the spread of the predictions about it. A curve describes the model, not the atmosphere; correlated predictors (temperature and boundary-layer height, say) are varied one at a time, into combinations the record may never contain.

3.9 Synthetic control

A synthetic control estimates the effect of an intervention on one *treated* unit (a site, a city) from a panel of units observed before and after a *cutoff* date. Units the intervention did not touch — the *donor pool* — are combined into a synthetic version of the treated unit that tracks it before the cutoff; after the cutoff, the synthetic series is the counterfactual, and

$$\text{effect}_t = \text{observed}_t - \text{synthetic}_t. \tag{3.6}$$

The average effect over the post-cutoff period is reported as the ATT (average treatment effect on the treated). For an air-quality outcome, a de-weathered series is the natural input: the weather is then not part of what the donors must reproduce.

3.9.1 Estimators

`scm` Augmented SCM (the default). At each date a ridge regression across the donors predicts the outcome from each unit's pre-cutoff trajectory; the synthetic value is the treated unit's ridge prediction plus a weighted sum of the donors' ridge residuals, the weights summing to one (non-negative unless allowed otherwise).

`abadie` Classic SCM: non-negative donor weights summing to one, fitted to the treated unit's pre-cutoff path. Interpretable, but it cannot reach a treated unit that lies outside the range of its donors.

`did` Difference-in-differences: a two-way fixed-effects baseline that assumes parallel trends.

`mcnnm` Matrix completion with nuclear-norm minimisation (Athey et al., 2021): the post-cutoff treated cells are treated as missing and filled from a low-rank fit of the panel.

`robust` Robust SCM (Amjad, Shah and Shen, 2018): the donor matrix is de-noised by singular-value thresholding before the weights are fitted.

`mlscm` ML-augmented SCM (*experimental*): a LightGBM or H2O model maps donor outcomes to the treated outcome in the pre-period. It lacks the causal guarantees of the others and warns so.

`bayesian` Bayesian simplex SCM: a flat Dirichlet prior on the weights, sampled by Markov chain Monte Carlo, giving posterior credible bands directly.

3.9.2 Diagnostics

`nm_scm_diagnostics` reports the pre-cutoff fit (`pre_rmse`, `pre_mae`, `pre_mape`, `pre_r2`), the post-cutoff effect (`att`, `att_cum`, `post_rmse`) and how concentrated the weights are: the Herfindahl index $\text{HHI} = \sum_j w_j^2$, the effective number of donors $1/\text{HHI}$, and the top donors with their share.

Caution. A good pre-cutoff fit is evidence only if the estimator could not have fitted any series as well. Every estimator that weights donors to match the pre-period overfits when the donors are many for the pre-cutoff dates, and augmented SCM (`scm`) most of all: its ridge features are the units' pre-cutoff outcomes, so at a pre-cutoff date the treated unit's own value at that date is one of the features it is predicted from, and with as many donors as dates it reproduces almost any pre-period. On the ULEZ panel of Chapter 11 (27 pre-cutoff months), a treated series replaced by random noise is fitted with $R^2 = 0.06$ from 5 donors, 0.81 from 20 and 0.999 from 53. So the SCM Studio refits each design with the treated unit's pre-cutoff values shuffled in time, and reads the pre-cutoff R^2 only when that shuffled series cannot also reach 0.5 (Section 11.3). The `scm` estimator's donor weights balance only the ridge residuals, so their concentration does not describe which donors the synthetic unit is made of.

3.9.3 Inference

Placebo in space Each donor in turn is treated as if it had been treated, and given a synthetic control built from the *other* donors (the real treated unit is never a donor to a placebo). With the post-cutoff mean effect as the statistic, the permutation p-value is

$$p = \frac{1 + \#\{\, j : |\bar{e}_j| \geq |\bar{e}_{\text{treated}}| \,\}}{1 + J}, \tag{3.7}$$

for J placebo units; the smallest it can be is $1/(J+1)$. The placebo effects also give a pointwise band for the treated effect.

Placebo in time The cutoff is moved to earlier dates (every `placebo_every` steps, keeping at least `min_pre_period` steps before it), where nothing happened. The true effect is ranked against the fake cutoffs' effects in the same way. Every fake post-period must end before the true cutoff.

Jackknife bands The fit is repeated leaving out each donor in turn; the spread of the effect gives a pointwise band. A result carried by one donor shows here.

Bootstrap bands The fit is repeated B times on random subsets of the donors (80 % by default), optionally with the pre-cutoff period resampled in blocks.

Conformal interval `nm_conformal_effect_interval` gives a finite-sample interval for the average post-period effect (Chernozhukov, Wüthrich and Zhu, 2021); `nm_rmspe_ratio_test` gives Abadie's post/pre RMSPE-ratio test.

Chapter 4

Getting started

4.1 The window before a table is open

normet opens on its **Data** tab with nothing loaded (Figure 4.1), the parameter panel folded away and the Assistant (Chapter 14) open beside the results. *Expand Panel* on the action bar, or Ctrl+B, brings the panel out; **View** → **Assistant** closes the Assistant. Under *Start with a time series* are two buttons — *Open Data…* and *Ask the Assistant* — and cards for the ways in: the examples, the five tables opened most recently (each with its folder, size and the date it last changed; one that has since moved is greyed out), and the two places data can come from without a file of your own, the Data Studio and the Transport Studio. A card with nothing to list is left out. Dropping a table file (Section 12.1) anywhere on the window opens it.

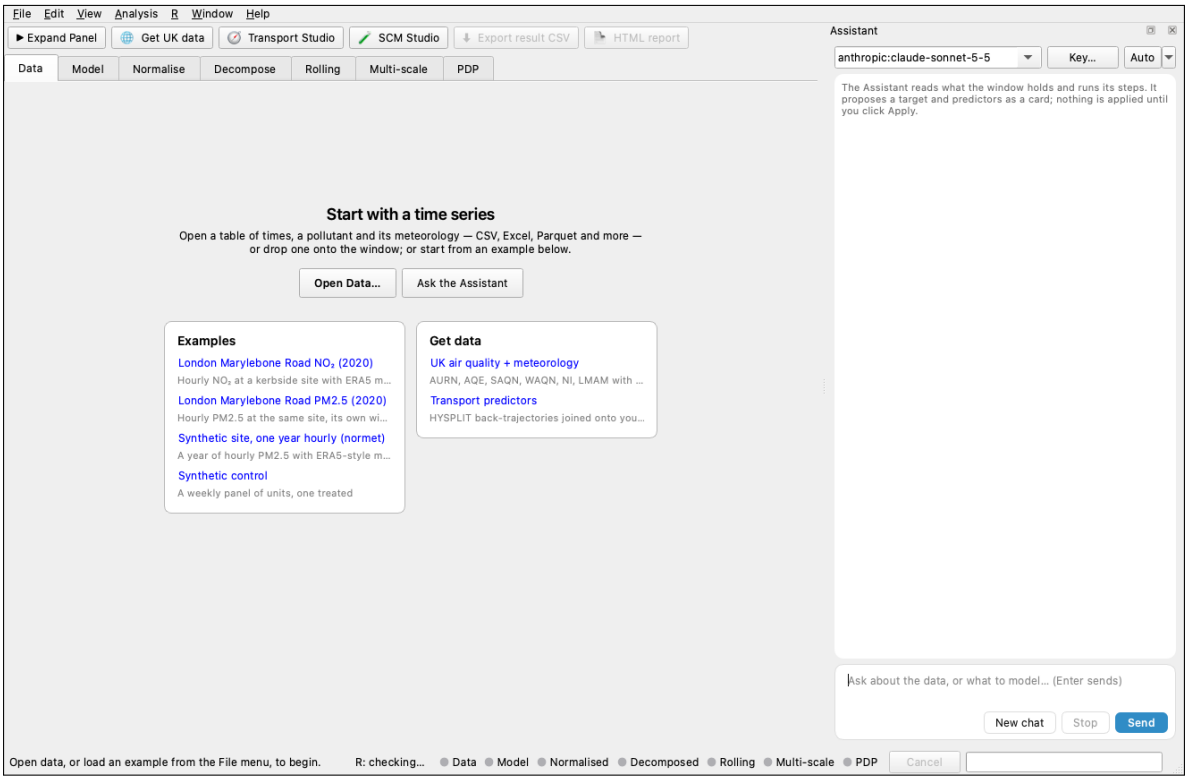


Figure 4.1: The window as it opens: the welcome on the **Data** tab, the action bar above the tabs (*Expand Panel* first), the Assistant on the right and the log at the bottom. The parameter panel is folded away until it is needed.

4.2 The examples

The examples are under **File → Load Example** and on the welcome's *Examples* card; each opens with its target and predictors set as its source sets them.

London Marylebone Road NO_2 (2020) The normet paper's de-weathering case study: hourly NO_2 at a kerbside site with ERA5 meteorology, January to August 2020, the UK's COVID-19 lockdown in it (`my1` below). This manual's main-window figures are made from it.

London Marylebone Road $PM_{2.5}$ (2020) The paper's transport-aware case: hourly $PM_{2.5}$ at the same site, its own wind and temperature with ERA5's other fields (`my1_pm25`).

Synthetic site, one year hourly A synthetic year of hourly data generated on the fly: $PM_{2.5}$, PM_{10}, NO_2, O_3, SO_2 and CO with ERA5-style meteorology (`u10`, `v10`, `t2m`, `d2m`, `blh`, `sp`, `ssrd`, `tcc`, `tp`, `rh2m`, `ws`, `wd`). The pollutants mix a seasonal cycle, a rush-hour cycle and a dependence on wind speed, so every step has something to show. The target is set to $PM_{2.5}$ and the meteorology ticked.

Synthetic control panel Opens the SCM Studio with its own example; the SCM Studio's **File → Load Example** has two (Chapter 11): the paper's London ULEZ panel and a synthetic weekly panel of 31 units, in which the unit "2+26 cities" has its SO_2 cut by about 35 % from 2015-10-23 and every other unit is a clean donor.

The real records of the normet model-description paper ship with the package, as its data sets (in a script, `my1` once `library(normet)` is loaded):

Data set	Contents	Used for
`my1`	Hourly NO_2 + ERA5 met, London Marylebone Road, Jan–Aug 2020	de-weathering
`scm`	Monthly de-weathered NO_2, 104 UK sites, 2016–2021	synthetic control (ULEZ)
`my1_pm25`	Hourly $PM_{2.5}$ + met at MY1, Jan–Aug 2020	transport-aware case
`traj_my1`	6-hourly HYSPLIT trajectory features arriving at MY1	transport-aware case
`extdata/traj`	Two days of raw HYSPLIT `tdump` files (`system.file("extdata", "traj", package = "normet")`)	Transport Studio

Sources: UK AURN (Defra, Open Government Licence v3.0); ERA5 (Copernicus Climate Change Service); HYSPLIT (NOAA ARL).

4.3 A first analysis

1. **Load data.** *Open Data…* and choose a table with its times, the pollutant and its meteorology (Chapter 12); set the *Time zone* the site keeps (Section 12.2). Bring the parameter panel out with *Expand Panel* (Ctrl+B), or ask the Assistant.
2. **Choose the columns.** Check the target and tick the predictors (Section 6.1).
3. **Train** and read the **Model** tab: is the held-out fit good enough to normalise with? (Chapter 6)
4. **Normalise** and compare the normalised series with the observed (Chapter 7).
5. **Decompose**, **Rolling** and **Partial dependence** as the question needs (Chapters 8–10).
6. **Keep the result:** export the tables, an HTML report and the session as a script (Chapter 13).

Chapter 5

The interface

The main window has a *parameter panel* on the left and *result tabs* on the right. The panel has a page for each step of the workflow, chosen from the bar at its top — **Data**, **Model**, **Normalise**, **Decompose**, **Rolling** and **PDP** — and each step has its own *Run* button beside the settings that govern it. Results land in the tab of the same step, which comes to the front when the step finishes. The panel starts folded away, leaving the width to the results and the Assistant; *Collapse / Expand Panel* on the action bar (Ctrl+B) folds it and brings it back. The window can be narrowed to about 740 pixels with the panel folded: the action bar then wraps onto a second row.

5.1 The parameter panel

Its pages, and what each holds (a page's first box goes untitled, the tab naming it):

Data *Open Data…*, the file's name with its rows and columns, the *Time zone* its times are kept on and how they were read (Section 12.1); then *Columns*: the target and the predictors (Section 6.1).

Model Backend, split, training fraction, search trials (or, for H2O, the time budget and the algorithms), seed (Chapter 6).

Normalise The variables resampled, the number of samples, CPU cores and the quantile band (Chapter 7).

Decompose Emission or meteorology; for meteorology the attribution and the groups (Chapter 8). Under it *Multi-scale decomposition*: three window widths (Section 9.2).

Rolling Window, step and samples (Section 9.1).

PDP Variables and grid points (Chapter 10).

The panel is as wide as its widest page needs. A configuration of the panel is saved and read from the **File** menu (Section 13.6).

5.2 Result tabs

Tab	Shows	Filled by
Data	overview, cycles, wind and pollution roses, statistics; rows left out	loading
Model	verdict, statistics table, parity plot, feature importance	Train model
Normalise	observed and normalised series, the quantile band, their cycles	Run normalisation
Decompose	the emission or meteorological components, the cycles	Run decomposition
Rolling	the mean of the rolling windows, with their spread	Run rolling normalisation
Multi-scale	the scale-space cascade and the energy in each band	Multi-scale
PDP	one partial-dependence panel per variable	Compute PDP

A tab with a judgement to make opens with a *verdict banner* in green (good), amber (caution) or red (not usable), followed by plain lines of what was run and, where there is one, a compact table. Every figure has a toolbar of small icons in the window's colours — home, back, forward, pan, zoom, subplot spacing, axes and curve options, save — then *For print...* (Section 13.3) and, at its right end, the value under the pointer. A figure is saved on a white page with dark ink whatever the window's appearance. Every figure is drawn as journals ask for one: 7 to 8.5 pt text, thin axes with ticks outward, legends without a frame, colours a colour-blind reader can tell apart, formulas with their subscripts (NO_2, $PM_{2.5}$), and a date axis written once at each level — the year once, then the months — with panels stacked on one time axis dated on the bottom panel only.

Resolution. The time series on the **Data**, **Normalise**, **Decompose**, **Rolling** and **Multi-scale** tabs have a *Resolution* selector above the figure: *As recorded*, *Hourly*, *Daily*, *Weekly*, *Monthly* or *Yearly*. A coarser step redraws the figure from the means of each period, which makes a long hourly record readable; steps finer than the record's own are greyed out. A record finer than daily and longer than 2,000 points opens as daily means — months of hours drawn as they are are a solid band — and the choice made on a tab is kept for its next result. Only the display changes: the result, its table and its export stay at the record's resolution. A series is drawn broken where the record has a gap (an interval over five times its usual step), not joined across it by a straight line. The σ a figure quotes are the record's, whatever step it is drawn at.

When both an emission and a meteorological decomposition have been run on the model, the **Decompose** tab keeps both: a *Method* selector beside *Resolution* switches between them, and a saved session brings both back.

5.3 The action bar

Above the tabs: *Collapse / Expand Panel*; the studios side by side — *Get UK data* (the Data Studio, Section 12.3), *Transport Studio* (Section 12.4) and *SCM Studio* (Chapter 11); then what is done with a result — *Export result CSV* (the current tab's table) and *HTML report* (the current tab's result as a report with its provenance).

5.4 Run buttons

Button	Needs	Runs
Train model	a table, one predictor ticked	`nm_build_model`
Run normalisation	a model, one variable to resample	`nm_normalise`
Run decomposition	a model	`nm_decompose`
Run rolling normalisation	a model	`nm_rolling`
Multi-scale decomposition	a model, a normalisation	`nm_rolling` ×3
Compute PDP	a model, one variable ticked	`nm_pdp`

A button is enabled only when it can act, and its tooltip says what is missing when it cannot. One task runs at a time, on a background thread, so the window stays responsive; while it runs the run buttons are disabled, *Cancel* is enabled, and the status bar names the task with the time it has taken so far. Where the task says how far it has come — R reports the Monte-Carlo batches, the rolling windows, the decomposition's steps, LightGBM's trials and the receptor times HYSPLIT runs; for H2O it is the time against its budget — the progress bar fills with it, and the line gives the share done and, once there is enough to go on, the time left; elsewhere the bar is a busy bar. When the task ends, the line its step writes says how long it took. Cancel stops the task: the R process doing it is ended, with whatever it started (a HYSPLIT run, say),

and the window is free at once. Each step's call is added to the command log when it succeeds (Section 13.2).

A step asked for before what it needs — normalising before a model is trained, a multi-scale run before the normalisation — is answered by a *notice* above the results, not a dialog: it says what is missing and carries the step that answers it on a button (*Train model*, *Run normalisation*); × puts it away, and it goes by itself once the step has been taken. At the right of the status bar a light per step — Data, Model, Normalised, Decomposed, Rolling, Multi-scale, PDP — turns green when that step has a result.

5.5 Menus and shortcuts

Menu	Command	Shortcut
File	Open Data...	Ctrl+O
	Load Example (the examples, SCM's)	
	Get UK Data (AQ networks + met)...	Ctrl+D
	Open Recent (eight tables)	
	Save Results... / Load Results...	Ctrl+S
	Reopen Last Session	
	Load Session Script...	
	Methods Paragraph...	
	Export Current Result...	Ctrl+E
	Export HTML Report: English, Chinese	
	Save Session as Script...	
	Save Config... / Load Config...	
	Run History...	
View	Command Palette...	Ctrl+K
	Collapse / Expand Panel	Ctrl+B
	Log..., Command Log...	
	Assistant	Ctrl+Shift+A
	Appearance: Follow System, Light, Dark	
	Language: Follow System, English, Chinese	
Analysis	Synthetic Control (SCM) Studio...	Ctrl+Shift+S
	Polar Plot...	
R	Locate Rscript..., Check R environment, Save Figure as R Script...	
Help	User Manual	F1
	Keyboard Shortcuts...	
	Documentation, About Normet	

On macOS, Ctrl is the Command key. The *command palette* (Ctrl+K) lists every menu command and every result tab; type a few letters of one and press Return. **Help → Keyboard Shortcuts** lists every shortcut, read from the menus; **Help → User Manual** (F1) opens this manual in the window's language.

5.6 The log and the Assistant

View → Log... opens normet's log in a window of its own, which can stay open beside the main window: each call into R (`R> nm_normalise(...)`) and R's own lines — what each step did, how many rows, LightGBM's search trials, how many windows a rolling run made. **View → Command Log...** opens the calls that repeat the session (Section 13.2). When a task fails, R's error is written to the Log, with the traceback, and shown in a dialog. The *Assistant* takes the right of the window, its whole height (Chapter 14).

5.7 Appearance and language

The window and its figures follow the system's light or dark appearance; **View** → **Appearance** pins one. A figure on a dark window is recoloured for display only — its faces, text, ticks and grid; colours that carry data are left alone — and saved in its light colours.

The window speaks English or Chinese: the system's language by default, or the one **View** → **Language** chooses, kept for the next session. The switch is immediate — menus, buttons, tabs, tooltips, verdicts and the figures' titles, labels and month names are written again in the language chosen, and a figure is saved as it is shown. The data's own words (column names, site codes), the log, the command log's code and the Assistant's conversation stay as they were written. What the system draws itself stays in the system's language whichever the window is written in: the open and save panels, their titles too, and, on a Mac, the items macOS adds to the menus (*AutoFill*, *Start Dictation*, *Emoji & Symbols*). An HTML report (Section 13.2) is written in the language picked as it is exported — **File** → **Export HTML Report**, or the *HTML report* button, lists each language named in itself — its headings, labels and figure; the language is kept for the next report, and is the system's until one is picked.

Chapter 6

Training a model

6.1 The table and its columns

A table opened in the main window needs a column called `date` (any case) whose values parse as dates; every other column numeric is offered as a target and as a predictor. The target is preset to the first column named like a pollutant ($PM_{2.5}$, PM10, NO_2, O3, SO2, CO, NOx), and the predictors to the columns recognised as meteorology:

```
u10 v10 d2m t2m blh sp ssrd tcc tp rh2m ws wd temp rh pressure
```

(matched without regard to case). *All* ticks every predictor — and, once all are ticked, unticks every one — and *Met only* ticks the meteorology alone. The target is never a predictor, even when ticked. The four calendar variables are added to every model automatically and are not listed.

Note. A meteorological column under another name is not recognised and must be ticked by hand — in MY1, `atmos_pres`. The same applies to the **Normalise** page's *Met only* (Chapter 7).

The **Data** tab has four pages. **Overview** shows the record at a glance: drag across the plot to pick a span, or click it to pick the point under the cursor (the period it stands for, where the series is averaged), and the rows are listed below the plot; with Shift held (or Ctrl, or Cmd) a drag or a click adds to the pick, a click on a picked point drops it, and Escape clears them. One line under the plot says the pick — its rows, and how many of them are left out — at its left, and the time, the variable and the value under the cursor at its right. In the list, rows are selected as in a spreadsheet — Ctrl- or Cmd-click for rows apart, Shift-click or a drag down the rows for a run — and right-clicked, on them or on their numbers, to exclude or restore them (a row right-clicked that is not selected is selected alone first). A right-click on the plot offers the same for every row picked, a point outside the pick picked first; *Exclude picked* and *Restore picked*, on the plot's *Resolution* row, do the same for the rows selected, or every row picked when none is. The list stays, its excluded rows in grey, to go on from. The *Exclude* box beside them takes spans typed one to a line (`2020-03-23 .. 2020-04-05`, or a single date) and *Apply*. Rows left out are shaded on the plot and are left out of every step after: training, normalisation, decomposition and the rest. The Command Log writes them as `left_out` and `df_kept`, and the session keeps them. **Cycles** draws a column's diurnal, weekly and seasonal cycles; **Wind**, when the table has wind speed and direction, the wind rose, a pollutant's pollution rose and its polar plot; **Statistics** each column's values, missing share, mean, standard deviation, minimum, quartiles and maximum. All four read the rows kept.

6.2 Settings

Backend `lightgbm` (the default) or `h2o`. H2O needs Java and the `h2o` R package; **R** → **Check R environment** says whether it loads.

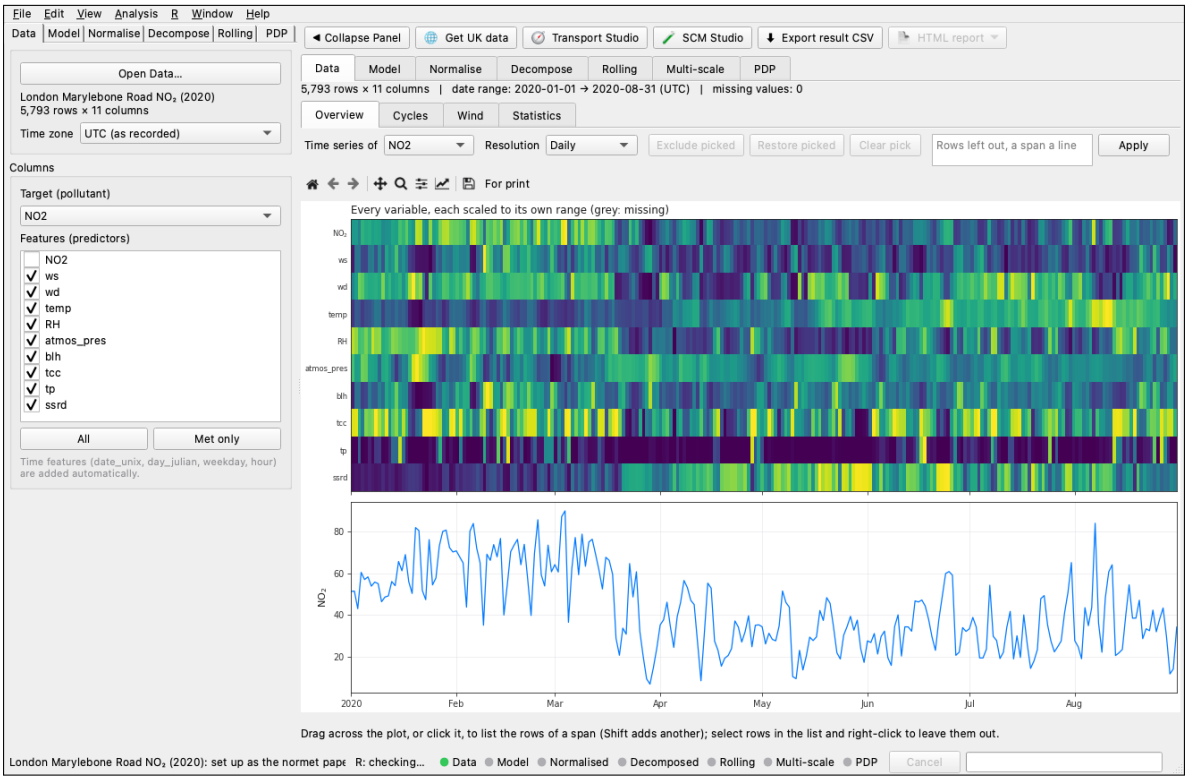


Figure 6.1: MY1 opened from **File** → **Load Example**: 5,793 hourly rows of 11 columns from 2020-01-01 to 2020-08-31, with no missing values. The example sets the paper's target, NO_2, and its nine meteorological predictors (`atmos_pres` among them, a name the window does not recognise). The **Overview** page draws every column as a heatmap, each scaled to its own range, above the series picked (here NO_2, daily).

Split method, Training fraction Section 3.1.1; 0.75 by default.
Search trials LightGBM's random-search trials, 20 by default. For H2O the row becomes *Time budget*, 60 s by default; each backend's value is kept when you switch.
Algorithms (H2O only) GBM alone by default; add more to widen the search.
Seed 7654321 by default: the split and the search are seeded with it.

6.3 Reading the Model tab

Train model runs `nm_build_model` — preparation, the split and the search — and fills the **Model** tab (Figure 6.2). The verdict is read from the held-out R^2: 0.7 or above is a good fit, 0.5 to 0.7 moderate, below 0.5 poor. The table is `nm_modStats` for the training rows, the testing rows and all rows (Section 3.2.1); the parity plot shows the held-out rows against the prediction, square, with the 1:1 line and, in its corner, the number of rows, the slope of predicted on observed (below 1: the extremes are pulled in) and the mean bias — as a density of hexagons where the rows run to thousands; the bars are the model's feature importance (each predictor's share of LightGBM's gain).

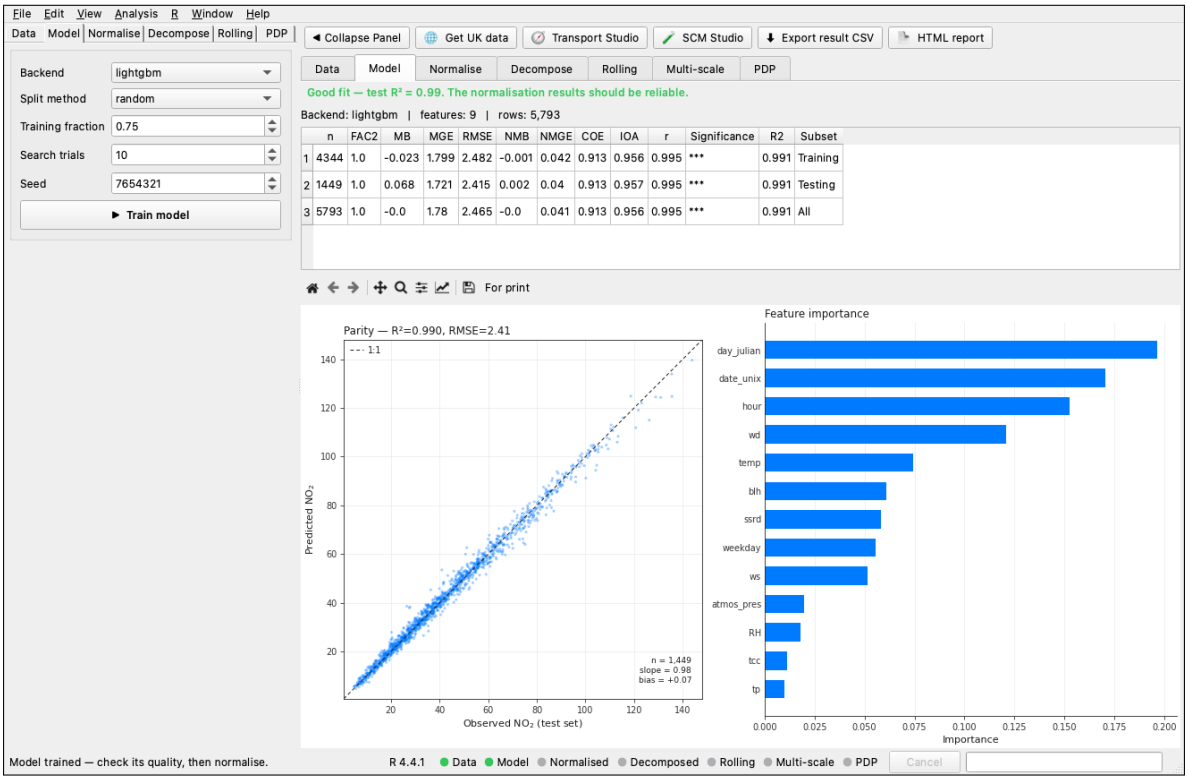


Figure 6.2: MY1 trained with LightGBM (10 search trials, random split): on the 1,449 held-out hours $R^2 = 0.991$ and RMSE = 2.42 $\mu g\,m^{-3}$, against 0.991 on the 4,344 training hours. The calendar — the day of the year, the trend and the hour — and the wind direction carry the most gain.

A fit on the training rows much better than on the held-out rows is expected of boosted trees and is not by itself a problem; a poor held-out fit is. Before normalising with a moderate or poor model, try more predictors, more search trials, or H2O. The normalised series inherits the model's errors.

Chapter 7

Normalising

Resample vars Every variable the model was trained on, the four calendar variables included. Ticked variables are resampled (their influence averaged out); unticked ones keep their observed values. After training the list is set to *Met only* — the usual de-weathering — and *All* ticks everything, which removes the trend and the seasonal and weekly cycles as well.

Samples Monte-Carlo draws, 300 by default (Section 3.3 on how many are enough).

CPU cores *auto* uses all but one.

5–95 % quantile band Also return the 5th, 25th, 75th and 95th percentiles of the draws and shade the outer band.

Beneath the series, the observed and normalised series are averaged by the hour of the day, the day of the week and the month, each mean with its 95 % confidence interval — the record's cycles, as openair's `timeVariation` draws them, and how much of each the weather moved. A cycle the record cannot show (the hours of a daily record) is left out; the cycles are averaged from the record as it is, whatever resolution the series above is shown at. In R: `nm_cycle_profiles(result, c("observed", "normalised"))`.

The line above the figure gives the number of timestamps, draws and resampled variables, and the mean of normalised − observed over the record: here $-0.20\ \mu g\,m^{-3}$, since over eight months the weather averages out. The monthly means tell the story the figure shows:

NO_2 ($\mu g\,m^{-3}$)	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug
observed	60.3	65.2	51.7	34.5	30.7	35.1	31.6	36.6
normalised	60.0	64.9	54.7	33.2	31.1	32.2	31.9	35.8

From February to April the normalised series falls by 31.7 $\mu g\,m^{-3}$ (49 %), against 30.7 $\mu g\,m^{-3}$ observed: almost none of the spring drop was the weather. In March the weather was cleaner than usual (observed 3.0 below normalised), in June and August dirtier.

Caution. The normalised series is the model's expectation under typical weather. A model that has not learnt the weather's effect — a poor fit, or meteorology missing from the predictors — returns something close to the observed series, which looks like a de-weathered series with nothing to remove. Read the **Model** tab first.

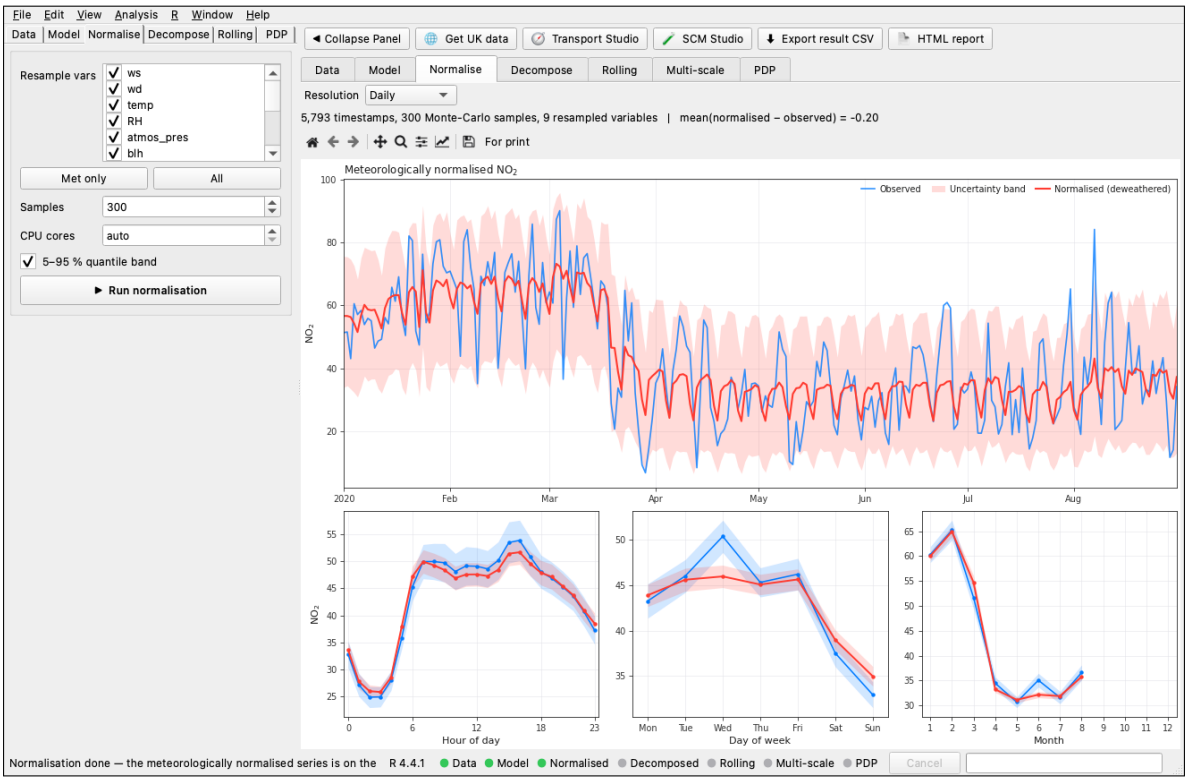


Figure 7.1: MY1 normalised with 300 draws of all nine meteorological variables, shown as daily means. The normalised series (red) falls in the third week of March, the lockdown, which the weather does not explain. Beneath, its cycles: the normalised series keeps the rush hours and the weekday–weekend difference — the calendar is held at its observed values — with less of the observed series' spread.

Chapter 8

Decomposing

8.1 By time scale: the emission decomposition

With *Method* set to *emission*, the decomposition runs the nested normalisation of Section 3.4 and plots the normalised series with its trend, seasonal, weekly, diurnal and short-term components, each with its standard deviation σ (Figure 8.1). The line above the figure names the component with the largest σ. On a record finer than daily, the diurnal and weekly components are drawn on the right as their cycles — the component's mean by hour of day and by day of week, shaded between its 25th and 75th percentiles — rather than as months of oscillation.

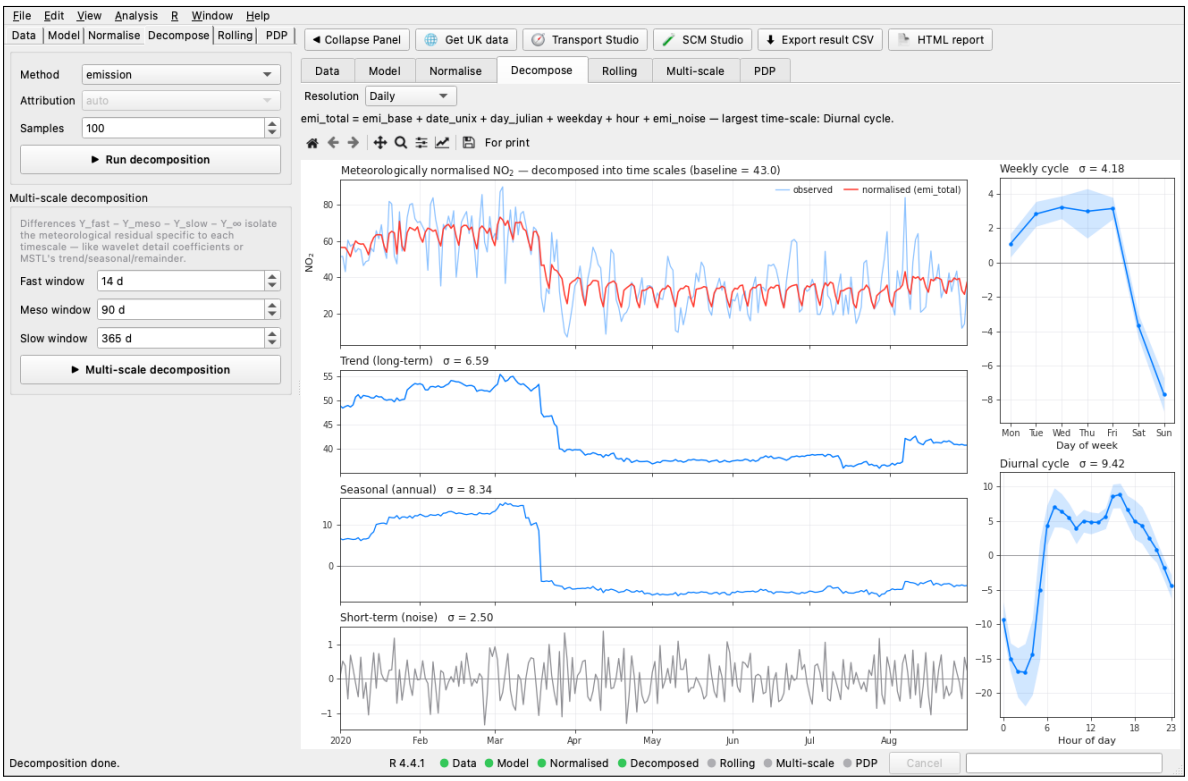


Figure 8.1: The emission decomposition of MY1 (100 draws per normalisation; baseline 43.0 μg m^{-3}). The lockdown appears as a step in both the trend ($\sigma = 6.59$) and the seasonal component ($\sigma = 8.34$): a one-off event is not a season, and the record is too short to tell them apart. The diurnal cycle (right) is the largest ($\sigma = 9.42$): low before 05:00, high through the working day, its spread wide because it shrank after the lockdown, as rush-hour traffic did; the weekly cycle ($\sigma = 4.18$) dips at the weekend.

8.2 By variable: the meteorological decomposition

With *Method* set to *meteorology*, the decomposition splits observed − normalised among the meteorological variables (Section 3.5). *Attribution* is enabled and *Groups* appears (it is hidden for *emission*):

Attribution *sequential*, *shapley*, or *auto* (Shapley when groups are given, else sequential).

Groups A table of the meteorological variables the model was trained on, each with its group: *Wind*, *Temperature and humidity*, *Boundary layer and pressure*, *Sky and rain*, or a name typed in. *Suggest groups* puts each recognised variable in its group; *Each alone* clears them. A variable left without a group is a contribution of its own. A group's rows are tinted in one colour, and the line under the table says how many contributions the grouping makes.

The figure shows met_total and one panel per variable or group, with a ranking of their σ beside `met_noise`, the part the model does not reproduce (Figure 8.2).

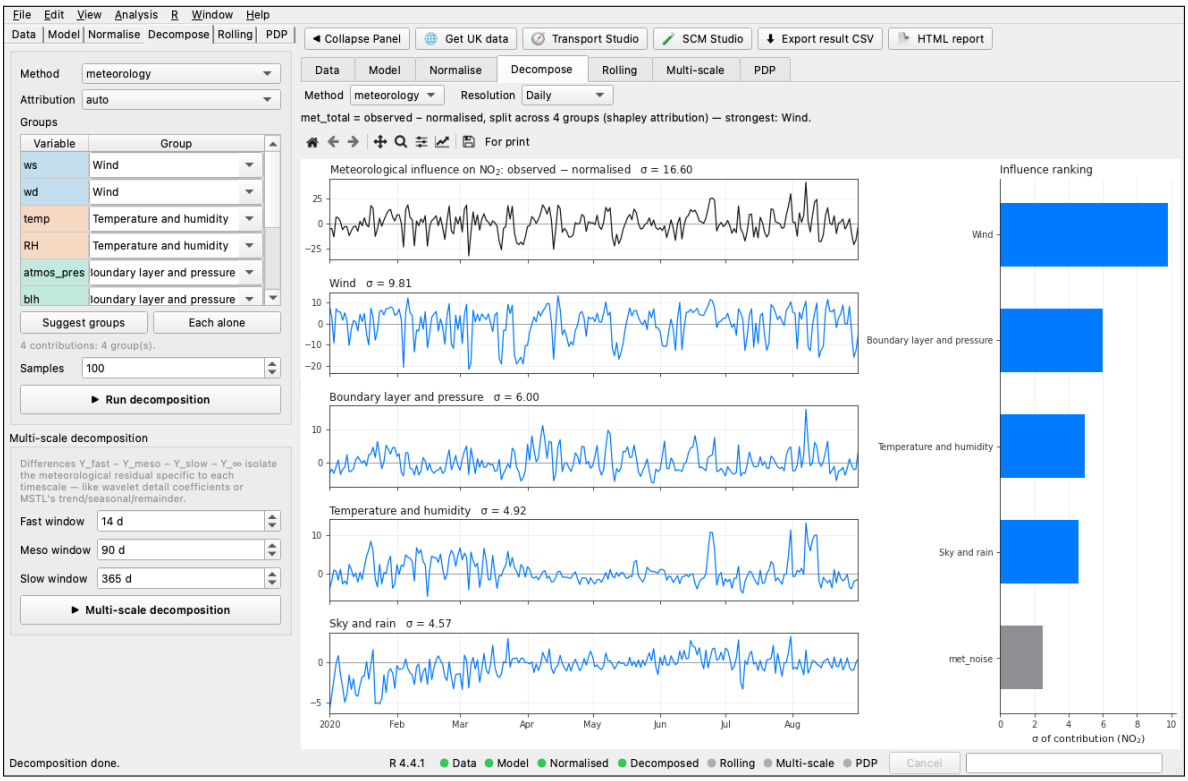


Figure 8.2: MY1's weather influence (observed − normalised, $\sigma = 16.60\ \mu g\,m^{-3}$) split among four groups by exact Shapley values (16 coalitions): wind (`ws`, `wd`; $\sigma = 9.81$), thermodynamics (`temp`, `RH`; 4.92), mixing (`blh`, `atmos_pres`; 6.00) and sky (`tcc`, `tp`, `ssrd`; 4.57).

The decomposition runs $k + 1$ normalisations sequentially or 2^k by Shapley, each of *Samples* draws, so its cost grows quickly with the number of variables. Ten variables by Shapley is 1024 normalisations: group them. Unlike the normalisation, the meteorological decomposition always resamples every meteorological variable trained on; it does not follow the **Normalise** page's list.

Chapter 9

Rolling and multi-scale

9.1 Rolling normalisation

Window (14 d), *Step* (7 d) and *Samples* (100) set the windows of Section 3.6; the variables resampled are the **Normalise** page's. The tab shows the mean of the overlapping windows at each time, with ±1 standard deviation across them where two or more overlap (Figure 9.1). The result table, exported, has one column per window.

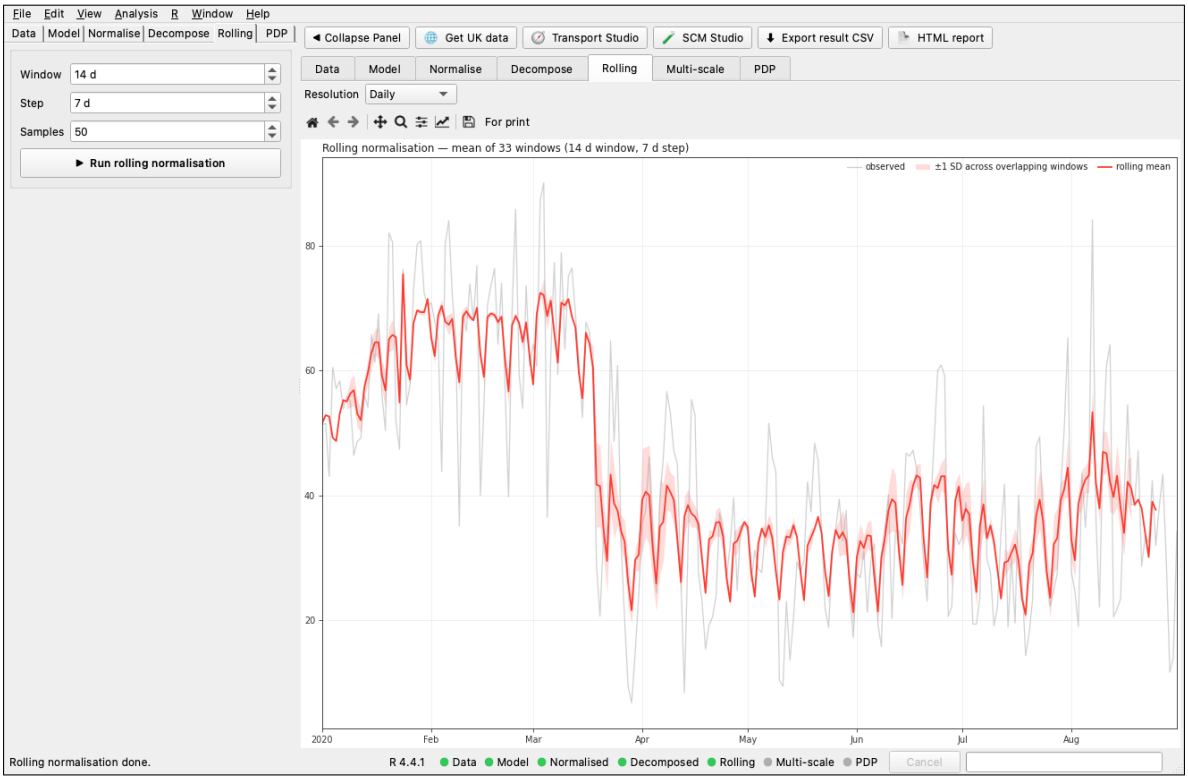


Figure 9.1: Rolling normalisation of MY1: 33 fortnightly windows started weekly, 50 draws each. Each window removes only the weather that varies within its own fortnight.

9.2 Multi-scale decomposition

The multi-scale box runs rolling normalisations at three widths — *Fast* (14 d), *Meso* (90 d) and *Slow* (365 d) by default — and differences them against each other and against the normalised series (Section 3.7); so the normalisation must have been run first. It uses the **Rolling** page's

Samples and costs about three rolling runs. The verdict names the band with the most energy (σ); a band that could not be computed is named in the notes, with why.

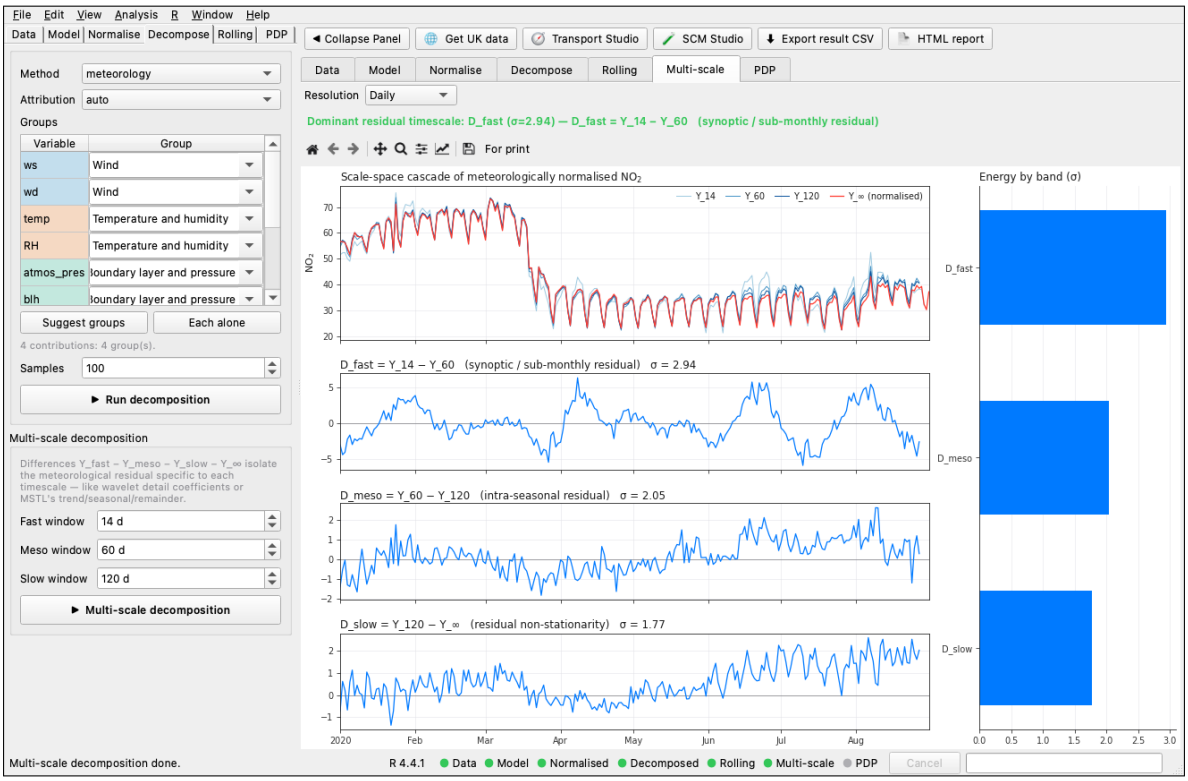


Figure 9.2: Multi-scale decomposition of MY1 at 14, 60 and 120 days (the 365-day default is longer than this eight-month record): 77, 13 and 5 windows. The synoptic band carries the most (D_{fast}, $\sigma \approx 3.0$), then the intra-seasonal and the slow.

Chapter 10

Partial dependence

After training, the variable list holds every predictor and the calendar variables, with the first four ticked. *Time variables* and *Met only* toggle those groups; *Grid points* (50) sets the resolution. Each panel shows the curve (`pdp_mean`) with a band of ± `pdp_std`, the spread of the predictions across the training rows at that value (Section 3.8). The short ticks along the bottom are the variable's deciles in the training data: where they are far apart — a curve's ends — the data are few. A variable of a few whole values (the hour, the weekday) is drawn as steps, a point on each value.

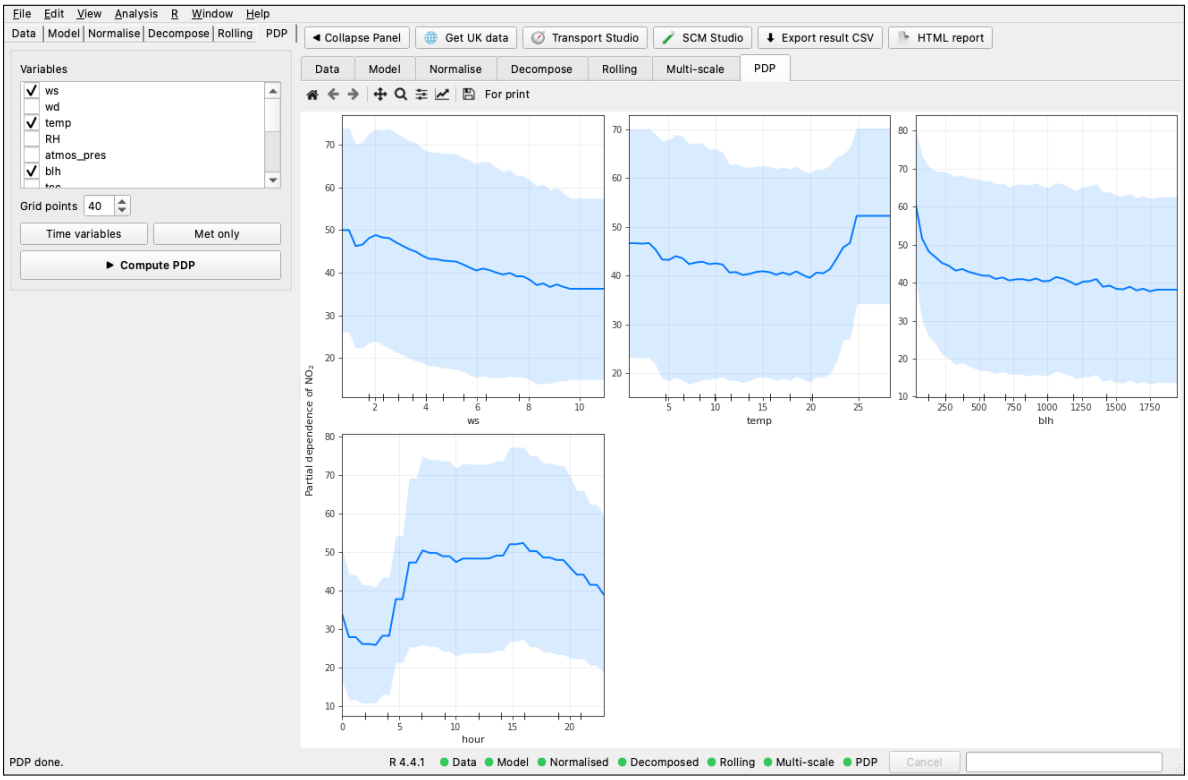


Figure 10.1: Partial dependence of the MY1 model: NO_2 falls as wind speed rises and as the boundary layer deepens from its shallowest, rises above about 20 °C, and follows the traffic day — low before 05:00, high from 07:00 to 17:00.

The bands are wide because the other predictors still vary across the rows; they are not a confidence interval for the curve.

Chapter 11

Synthetic control (SCM Studio)

The synthetic-control workflow takes a different input — a long panel of many units — so it has a window of its own, opened from *SCM Studio* on the action bar, **Analysis** → **Synthetic Control (SCM) Studio** (Ctrl+Shift+S) or the welcome's example card. It is laid out as the main window is: the panel on the left, its pages chosen from a bar — **Data** (the panel data and its columns), **Design**, **Estimator**, **Inference** — result tabs on the right (**Panel**, **Fit**, **Placebo Space**, **Placebo Time**, **Uncertainty**, **All Units**), and *Export current tab data* above them. The method is described in Section 3.9.

This chapter's figures reproduce the ULEZ case study of the normet paper: the London Ultra Low Emission Zone, launched on 8 April 2019, evaluated at Marylebone Road (MY1) on monthly de-weathered NO_2, with traffic sites elsewhere in the UK as donors. To keep COVID-19 out of the post-period the panel ends in February 2020, and only traffic sites with a complete record are kept: 53 donors over 38 months. The SCM Studio's **File** → **Load Example** → **London ULEZ** sets this design up; in a script:

```
library(normet)
df <- subset(scm, date >= as.Date("2017-01-01") & date < as.Date("2020-03-01"))
df <- df[grepl("Traffic", df$type), ]
counts <- table(df$code[!is.na(df$NO2_dw)])
donors <- setdiff(names(counts)[counts == length(unique(df$date))], "MY1")
df <- df[df$code %in% c(donors, "MY1"), ]
write.csv(df, "ulez_panel.csv", row.names = FALSE)
```

11.1 Panel data and columns

Open Data... (also **File** → **Open Data...**) reads a long table, in any of the formats of Section 12.1: one row per unit and date. *Use main-window data* takes the main window's table, when it has a unit column. The *Date* column is guessed from its name, the *Unit* column as the first non-numeric column with between 3 and a quarter as many distinct values as rows, and the *Outcome* as a numeric column whose name ends in `wn` (the de-weathered series of the paper's tables), else the first numeric one. Check all three.

11.2 Design

Treated unit The unit the intervention reached.

Cutoff date The first date of the post-period. The month containing the intervention is a judgement: here 2019-04-01, the ULEZ's launch month.

Donor pool Every other unit, ticked. Untick any unit the intervention could have reached, directly or by spill-over — a donor that was itself treated biases the counterfactual towards

the treated unit.

The **Panel** tab plots the outcome of every unit with the treated one in red, and checks the panel: amber when more than 10 % of the date × unit cells are missing or there are fewer than 10 pre-cutoff dates, green otherwise.

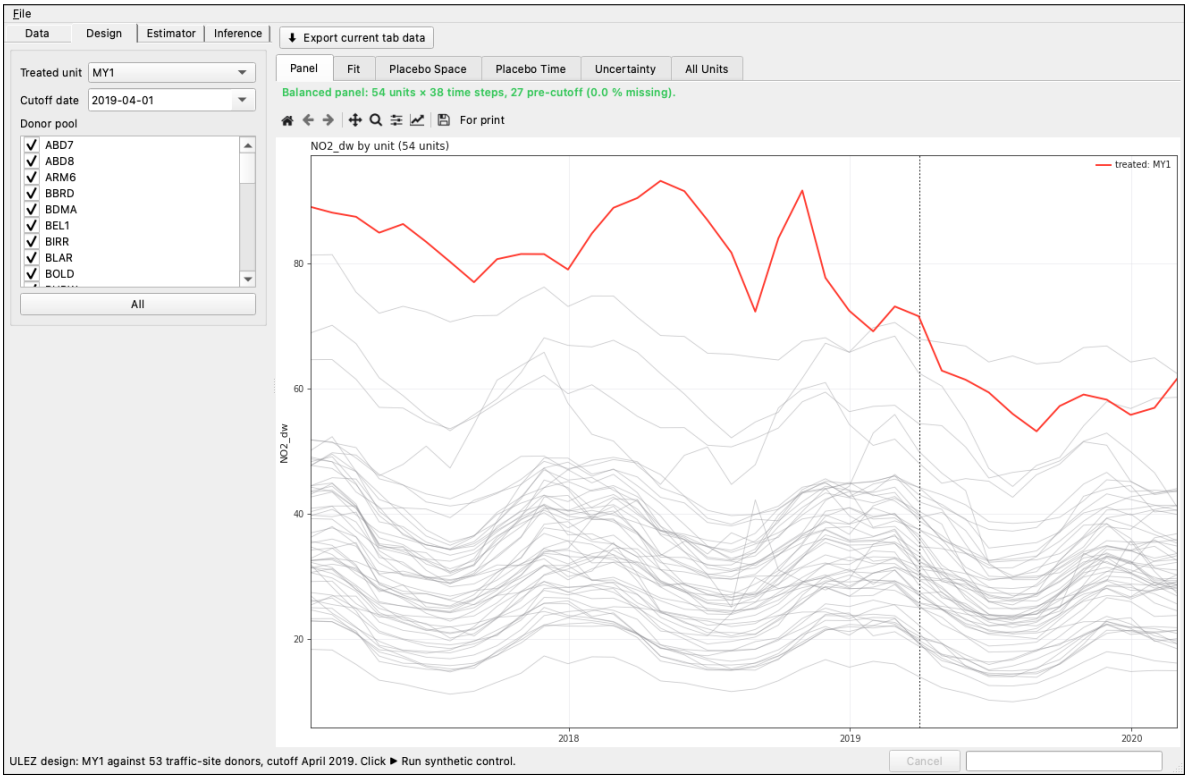


Figure 11.1: The ULEZ panel: MY1 (red) against the 53 traffic-site donors, monthly de-weathered NO_2, January 2017 to February 2020. MY1 is the highest of them throughout.

11.3 Fitting

Backend chooses the estimator (Section 3.9); *ML backend* (`lightgbm` or `h2o`) and *Search trials* (5; for H2O *Time budget*, 15 s) apply to `mlscm`, *Draws* (1000) to `bayesian`. *Run synthetic control* fills the **Fit** tab: observed and synthetic, the effect, the largest donor weights, and the diagnostics table.

For `scm`, `abadie`, `did`, `mcnnm` and `robust` the fit is followed by one more, of the same design with the treated unit's pre-cutoff values shuffled in time: a series with the treated unit's level and spread but no relation to the donors (Section 3.9.2). Its pre-cutoff R^2 is the table's `pre_r2, shuffled` row, and it decides how the verdict reads:

- If the shuffled series also reaches $R^2 \geq 0.5$, the design fits anything and the observed R^2 is not read. The verdict is amber, gives the ATT and both R^2, and sends you to Placebo in space and the other estimators.
- Otherwise the verdict is read from the pre-cutoff R^2: green at 0.8 or above, amber from 0.5, red below; it gives the ATT.

`mlscm` and `bayesian` are too slow to fit twice; their verdict is read from the pre-cutoff R^2 alone. An `scm` fit adds a line saying its weights balance the ridge residuals only.

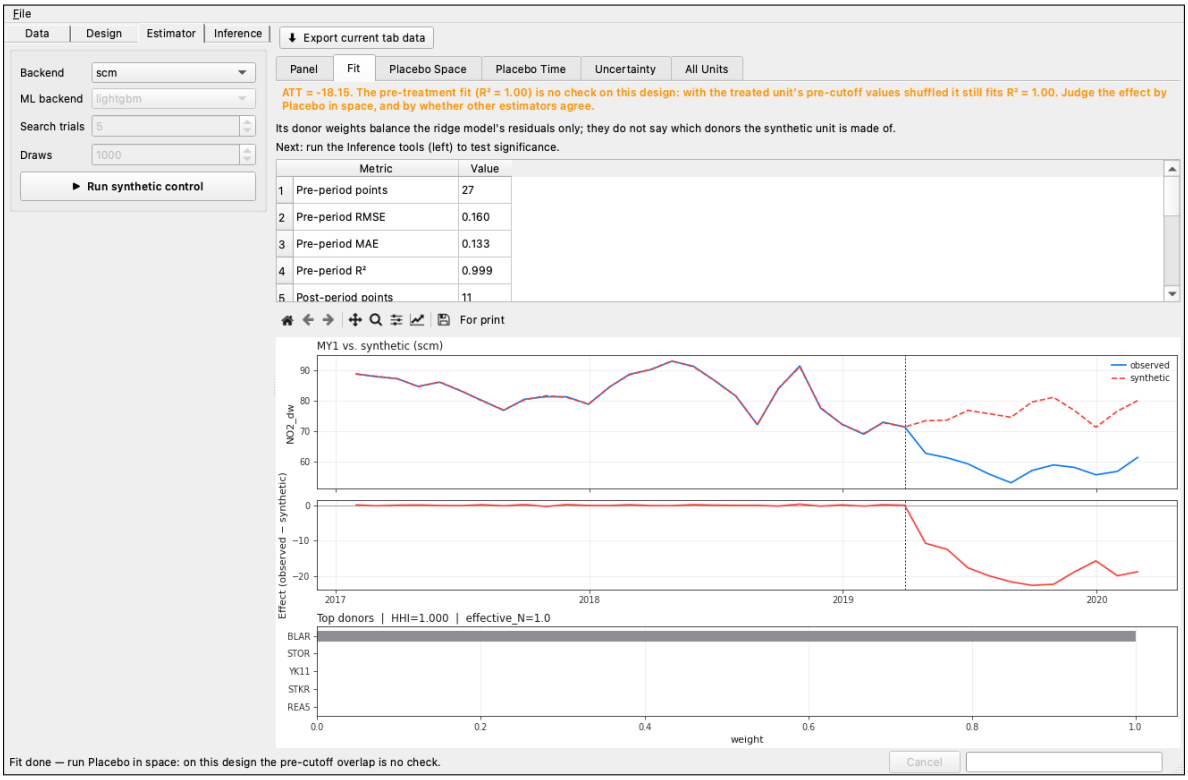


Figure 11.2: The augmented SCM (`scm`) fit for MY1: ATT $= -18.15\ \mu\mathrm{g\,m^{-3}}$ over the 11 post-cutoff months. Its pre-cutoff R^2 is 0.999, but with MY1's pre-cutoff values shuffled it still fits 0.9998 (both shown as 1.00), so the verdict does not read the fit and points to the placebo tests.

With 53 donors for 27 pre-cutoff months, this design is one where augmented SCM fits anything. Its top donor carries all of the weight (HHI = 1.00), but those weights only balance the ridge model's residuals.

The estimators do not agree on this design. Run on the same panel, cutoff and donors:

Backend	pre-cutoff R^2	shuffled R^2	pre-cutoff RMSE	ATT (μg m^{-3})
`scm`	0.999	1.000	0.16	−18.15
`abadie`	−2.94	−3.33	13.20	−6.83
`robust`	0.37	0.16	5.27	−9.48
`did`	−0.23	−0.09	7.36	−20.67

The shuffled column is the baseline the **Fit** tab computes: only for `scm` does a series unrelated to the donors fit as well as MY1, so only its pre-cutoff R^2 goes unread. Classic SCM cannot fit MY1 at all: its weights are non-negative and sum to one, so the synthetic unit cannot exceed the highest donor, and MY1 is above every donor. Robust SCM fits it only loosely. The four estimates agree on the sign and differ threefold in size. That is a finding about the design, not a fault of the software: a treated unit outside its donors' range is the case synthetic control handles worst, and an analysis of it should report the spread, not one number.

11.4 Inference

Run placebo in space The **Placebo Space** tab draws every donor's placebo effect in grey and the treated unit's in red, then the treated effect inside a 95 % band of the placebos. Green at $p \leq 0.05$, amber with "borderline" up to 0.15, amber otherwise.

Run placebo in time Fake cutoffs every *Placebo every* steps (7), each with at least *Min pre-period* steps (30) before it. A fake post-period must fit before the true cutoff, so this needs a long pre-period; the tab says so when none fits. Green at $p \leq 0.05$; when p is the smallest the number of fake cutoffs allows, the verdict says so and suggests a smaller step.

Uncertainty bands Jackknife (leave each donor out) or bootstrap (B refits, 100 by default). The verdict gives the mean post-cutoff effect and on what share of post-cutoff dates the band excludes zero; green at half or more.

Run all units Fits every unit as if it had been treated, with all the others as donors, and ranks them by the size of their post-cutoff effect. The treated unit should stand out: green when it is in the top tenth.

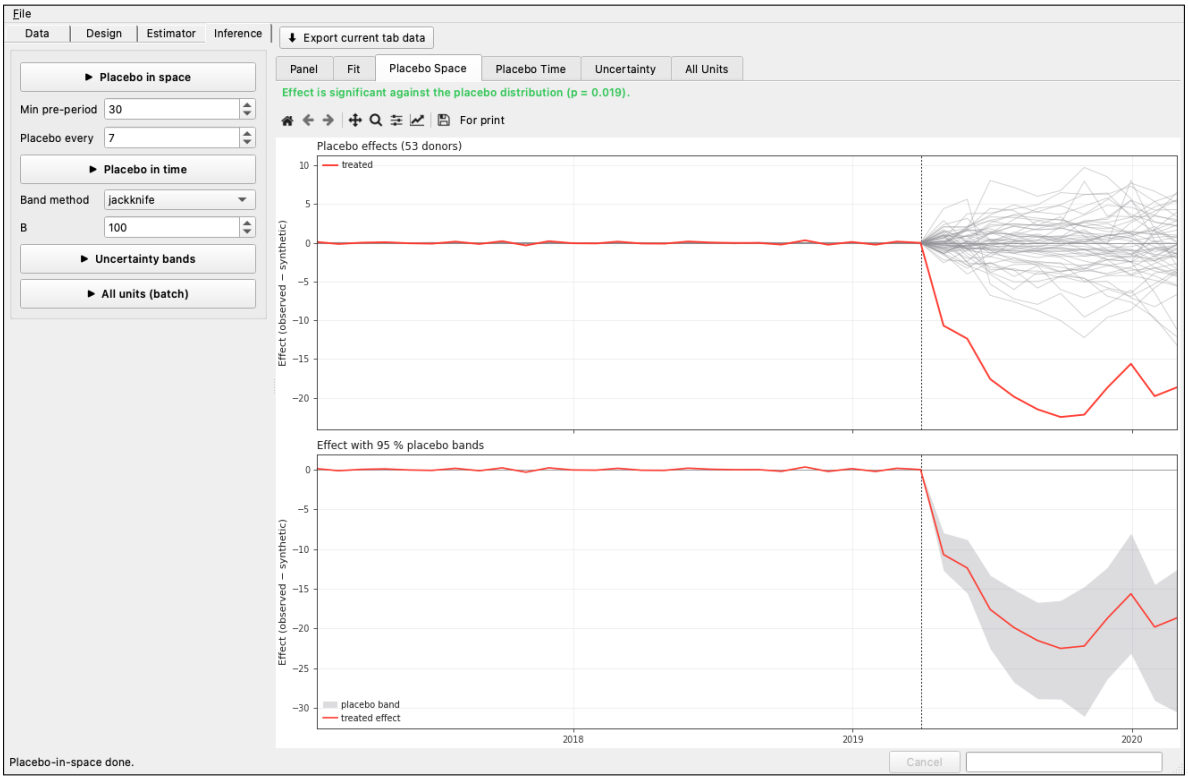


Figure 11.3: Placebo in space for the ULEZ: no donor's placebo effect comes near MY1's, so $p = 1/54 = 0.019$, the smallest 53 placebos allow. Every placebo is fitted by the same estimator, so this comparison is fair even where the pre-period fit is not informative.

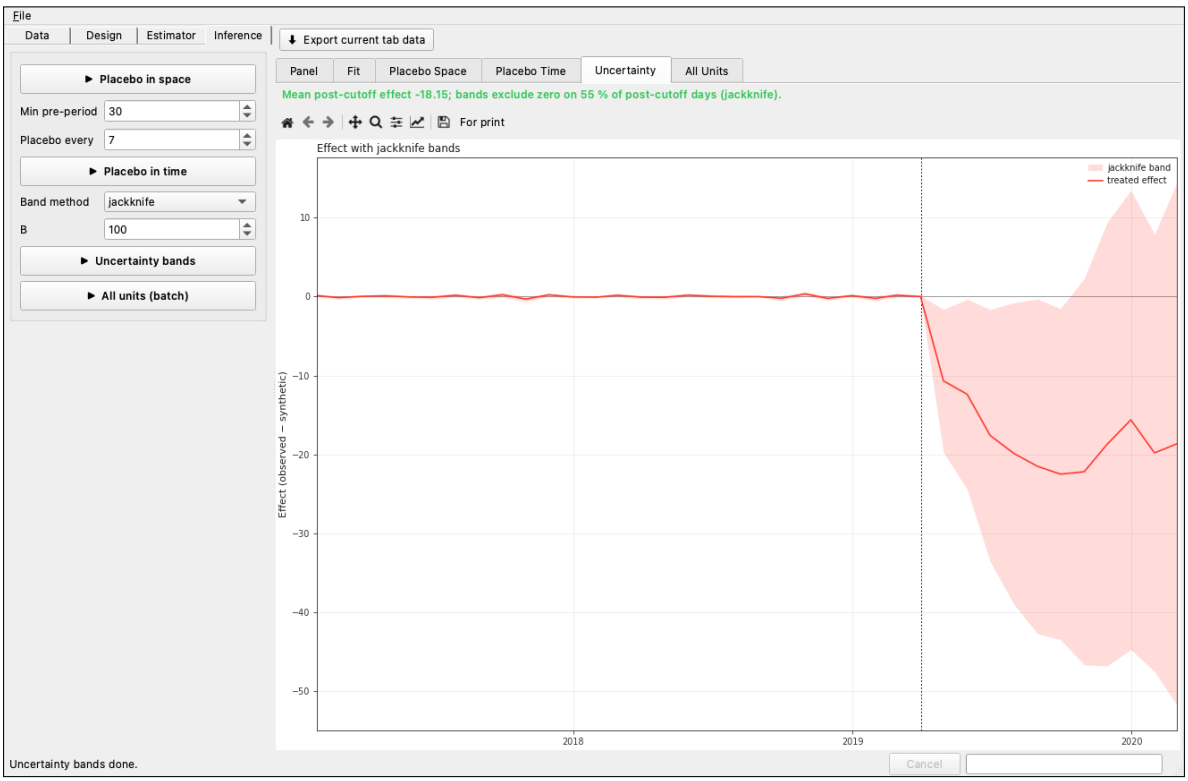


Figure 11.4: Jackknife bands: the effect is recomputed 53 times, leaving out one donor each time. The mean post-cutoff effect is $-18.15\ \mu\mathrm{g\,m^{-3}}$, and the band excludes zero on 55 % of the post-cutoff dates (the verdict says "days"; the panel is monthly).

Chapter 12

Getting data in

12.1 Input format

Open Data… (Ctrl+O) reads CSV and other delimited text (`.csv`, `.tsv`, `.txt`, `.dat`; comma, semicolon, tab, bar or space, a decimal comma, UTF-8 or a Windows code page — each told from the file), any of them gzipped, Excel (`.xlsx`, `.xlsm`, `.xls`), Parquet, Feather and JSON. The table needs:

- its times: a column named `date`, `datetime`, `time`, `timestamp` (any case, or their Chinese words), a date column with a time-of-day column beside it, or `year`, `month`, `day` (and `hour`) columns;
- the pollutant(s), numeric;
- the meteorology and any other predictors, numeric.

The times may be written in any of the usual ways: ISO 8601, `31/01/2020 13:00` (day first) or `01/31/2020` (month first) — the order told from the values, and said under the file's name when they do not tell —, `31.01.2020`, Chinese year–month–day writing, `2020013113`, Unix seconds or Excel's serial days. How they were read is shown under the file's name. One row per timestamp, hourly or coarser. A table of several sites in one file belongs in the SCM Studio, or in a script with `nm_do_all_multisite`. A table without times is refused, with a pointer to the SCM Studio for panel data. Missing values are allowed: rows without a target are dropped, missing predictors filled (Section 3.1).

12.2 The clock: time zone

The *Time zone* under the file's name is the clock the analysis keeps and its figures and tables show: the site's local time, or UTC (the default). Times written without an offset are taken to be on it, as a monitoring network's local-time record is; times written with one (`+08:00`, `Z`) are put on it. The hour and the weekday the model learns from are then the site's own — a rush hour stays at its hour through summer time. Meteorology and trajectories fetched on UTC (the Data and Transport Studios) are put on this clock as they are joined, so an hour of ERA5 meets the hour of the measurement it belongs to. The choice is kept for the next session; changing it reads the open file again. In R, `nm_read_time_series(path, zone = ...)` reads a table the same way.

12.3 Data Studio: UK air quality and meteorology

Get UK data (Ctrl+D) assembles a model-ready table without leaving the app (Figure 12.1):

1. **Air quality.** Choose a network — AURN (the national network, about 210 sites), AQE, SAQN, WAQN, NI or LMAM (locally managed), about 1500 stations in all — tick the pollutants ($PM_{2.5}$, NO_2 and O_3 by default), and *Find stations*. The **Stations** tab lists the sites that measure them; search by name or official code (`MAN3` is Manchester Piccadilly) and select one.
2. **Date range.** The archives are whole calendar years, from each station's opening; selecting a station snaps the range to its record.
3. **Meteorology.** Open-Meteo's ERA5 archive (no key), or the Copernicus CDS (needs the `ecmwfr` R package and a CDS key, set with `ecmwfr::wf_set_key`). Open-Meteo adds `t2m`, `d2m`, `rh2m`, `sp`, `tcc`, `tp`, `ssrd`, `ws`, `wd`, `u10` and `v10` — names the *Columns* box recognises as meteorology. They come on UTC and are put on the window's clock (Section 12.2) as they are joined.
4. **Fetch & merge** downloads the hourly measurements and the meteorology for the site's coordinates and joins them on the hour. **Preview** and **Table** show the result; *Save merged CSV...* writes it and *Send to main window* opens it there.

The archives are the openair `.RData` files R's `importUKAQ()` reads, read by the R package (`nm_fetch_ukaq_measurements`), so the data match an openair workflow. The Data Studio needs an internet connection; a table sent from it is held in memory, and the command log says so rather than writing a `nm_read_time_series` it could not repeat.

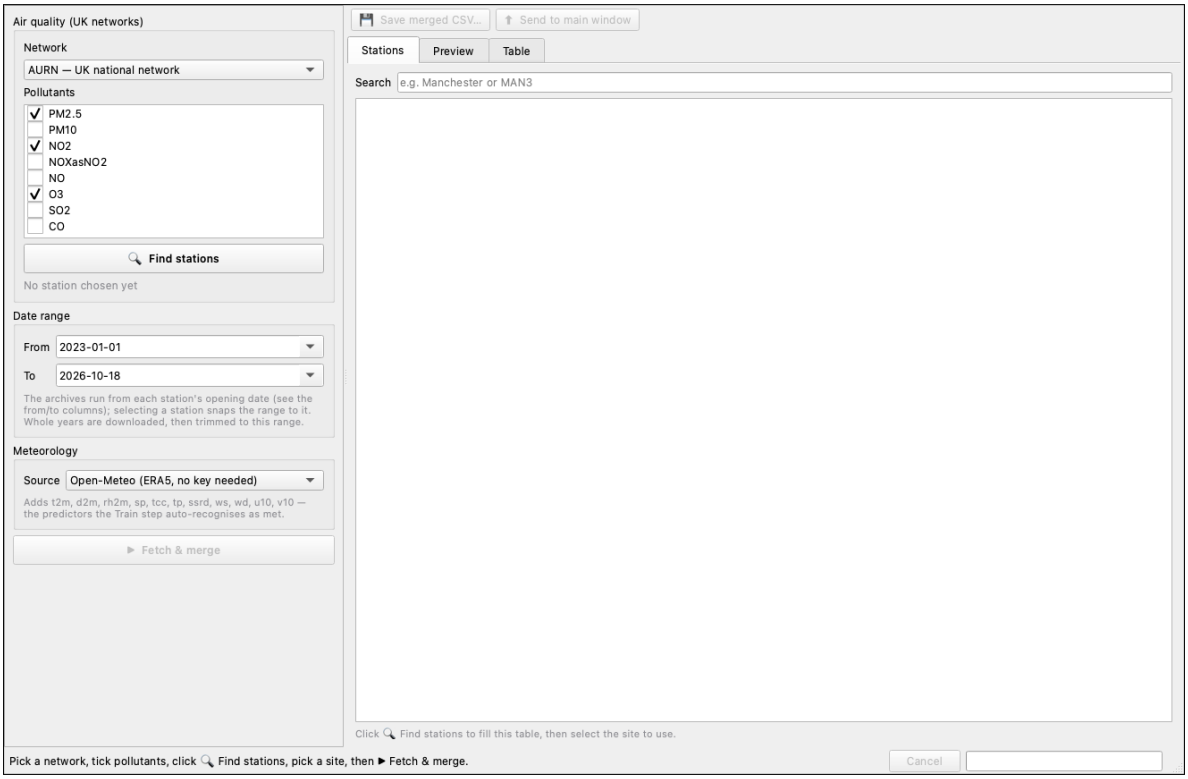


Figure 12.1: The Data Studio before a search: network, pollutants, date range and meteorology source on the left; the station table, preview and merged table on the right.

12.4 Transport Studio: back-trajectory predictors

Local meteorology does not say where the air came from. The Transport Studio turns HYSPLIT back-trajectories into predictors that do, and joins them onto the table (Figure 12.2). Its panel has two pages: *Trajectories*, for files already made, and *HYSPLIT*, for running them.

Trajectory files. *Add files…* takes existing HYSPLIT `tdump` files (no HYSPLIT installation is needed to read them). *Site name* tags the result with a `site` column, so batches for several receptors, built one at a time, each fill only their own site's rows. *Null trajectories shorter than* (72 h) blanks trajectories that stopped early because the meteorology ran out or the air left the domain, which would otherwise pass for short-range transport.

Source regions. Optional regions, as bounding boxes (name, lon/lat min and max) or polygons from GeoJSON or Shapefile files (`geo` extra). Each adds a residence-time feature: the fraction of the trajectory spent inside. Overlapping regions each count an endpoint inside both.

Features. Every trajectory becomes one row of: `traj_n_endpoints` and `traj_age_max_h` (quality: how far back it reached), `traj_dist_km` (straight-line reach), `traj_pathlen_km`, `traj_speed_kmh`, `traj_inflow_deg` (bearing from the receptor to the origin), `traj_height_mean` and `traj_height_min`, one `traj_resid_`*region* per region, and — where the `tdump` carries them — along-path rainfall, mixing depth, humidity, pressure and temperature.

HYSPLIT (advanced). With a local `hyts_std`, the studio can run the trajectories itself, on its *HYSPLIT* page, for a table of receptors, at a chosen height, hours back and time step, with GDAS1 meteorology it downloads or ARL files already on disk. The features are built with the source regions set on the *Trajectories* page.

Joining. *Send to main window* joins the features onto the open table: each row takes the latest trajectory that has *already arrived*, within one release interval — a nearest-time join would give 04:00 the 06:00 trajectory, air that has not yet arrived — and the new `traj_*` columns are ticked as predictors. The trajectories' times, UTC as HYSPLIT writes them, are put on the window's clock first (Section 12.2); the studio is given the table's times in UTC to start them from.

Caution. Trajectories released every six hours are constant over six hourly rows. Under a random split a tree model uses them as a time stamp and its test R^2 is inflated (Section 3.1.1). Judge a transport-aware model with `month_ts`. To split $PM_{2.5}$ into local-weather and transport contributions, give the meteorological decomposition two groups, `local` and `transport` (Section 3.5).

12.5 Polar plot: where a pollutant comes from

Analysis → Polar Plot… draws a variable of the open table by wind direction and speed, as openair's `polarPlot` does (Figure 12.3): its mean, median, 95th percentile or maximum in bins of 10° of direction and of speed, on a compass, north at the top. The wind columns are found by name (`ws` and `wd`), else made from `u10` and `v10`, and can be chosen. In a script: `nm_plot_polar(df, target = "NO2", ws_col = "ws", wd_col = "wd")` in R.

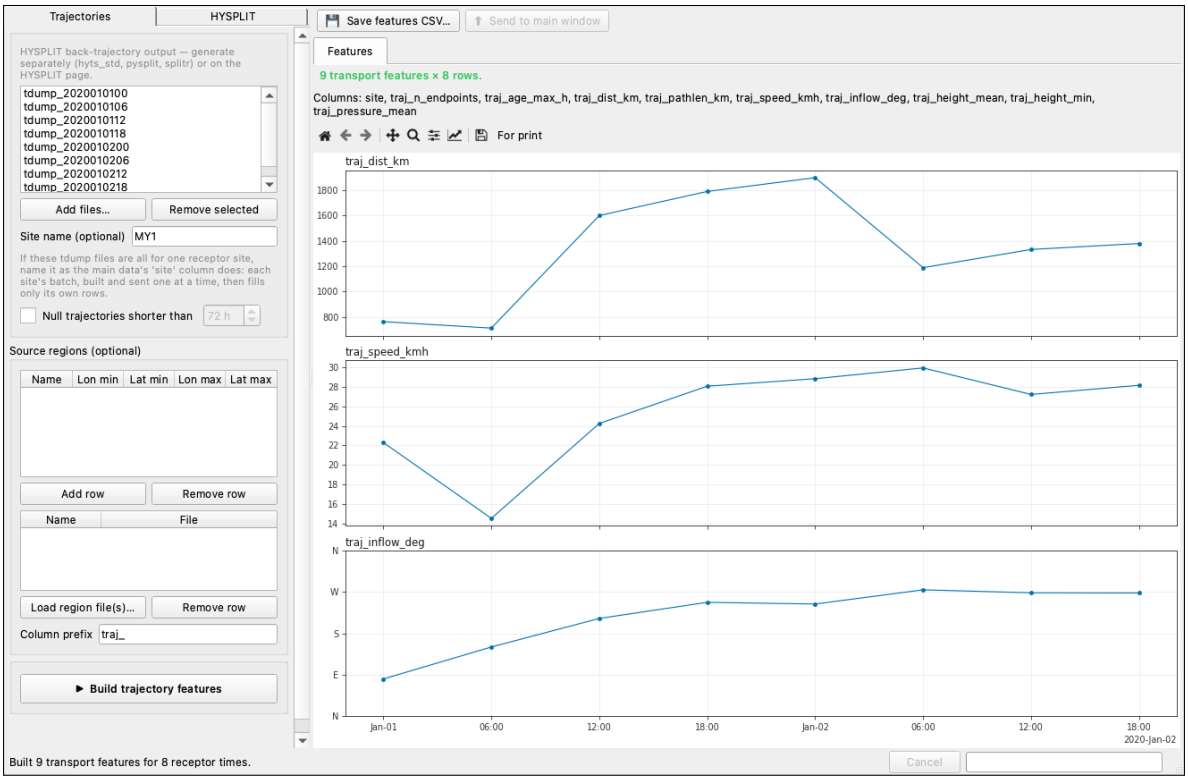


Figure 12.2: The two days of `tdump` files that ship with normet, built for receptor MY1: 9 features for 8 receptor times, released every six hours. Inflow turns from the east (80°) to the west (270°) over 1–2 January 2020.

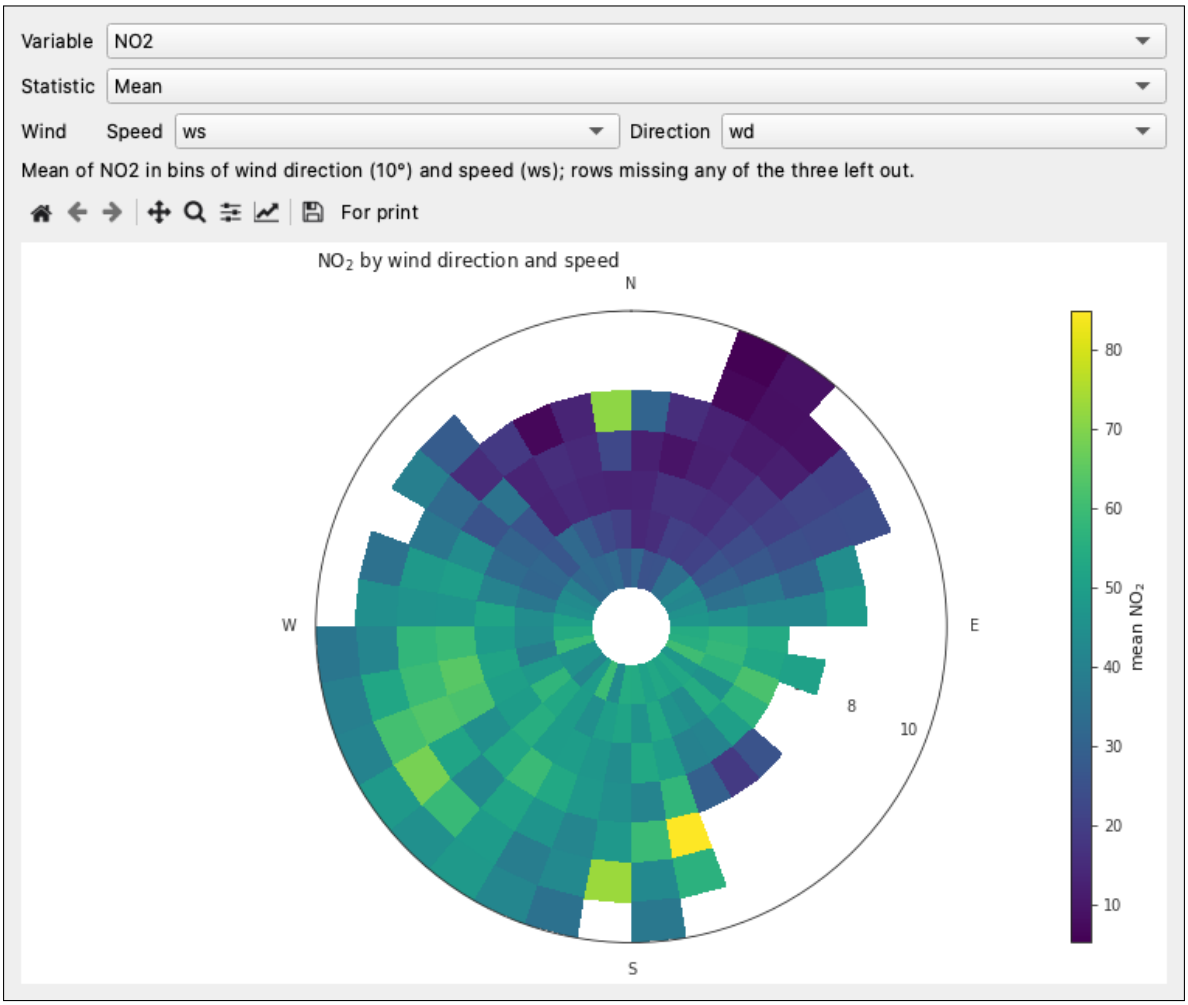


Figure 12.3: Mean NO_2 at MY1 by wind direction and speed: highest with winds from the south and south-west, lowest with winds from the north and north-east.

Chapter 13

Exporting and reproducing

13.1 Tables and reports

Export result CSV (Ctrl+E) The table behind the current tab: the loaded data on **Data**, the statistics on **Model**, the normalised series with its quantiles, the decomposition, every rolling window, the multi-scale series, the partial-dependence values.

HTML report (File → Export HTML Report, or the button; pick the report's language from its menu.) The current tab's result as one self-contained HTML file — figures embedded, no scripts or external links — with a summary of the model and the result, and its provenance: normet's version, the time, the platform and R version, the host and user names, the shape and hash of the data, the panel's settings and their hash, and the seed. Remove the host and user names before sharing a report if they should not travel with it.

13.2 The command log

Every step that decides a result — opening a table, training, normalising, decomposing, a rolling run, a multi-scale run, partial dependence — is written to **View → Command Log...** as the normet call that does the same, with the arguments the window actually used. An entry is written only once its task has succeeded, from the call that was made, not from what the panel shows now. *Copy* puts the log on the clipboard, *Save as Script...* (also **File → Save Session as Script...**) writes it as an R script that repeats the session, and *Clear* empties it.

The window draws its figures with matplotlib; the R package draws the same results with ggplot2. So each result's entry ends with the R package's figure of it — `fig_normalise <- nm_plot_normalise(normalised)`, and `nm_plot_decomposition_stack`, `nm_plot_rolling` and `nm_plot_pdp_grid` for the others — drawn where the script is run in R or RStudio (not under `Rscript`), and changed as any ggplot is. **R → Save Figure as R Script...** writes the result the open tab shows (Normalised, Decomposition — the method shown — Rolling or PDP) as a CSV, and beside it the R that reads it, draws it so and saves it as a PDF: a figure to change in R without training the model again.

The session behind this manual's figures saves as (the emission decomposition, the multi-scale and partial-dependence calls left out):

```
# 21:32:48 Load example: London Marylebone Road NO2 (2020)
df <- my1

# 21:33:07 Train model
res <- nm_build_model(df, target = "NO2", backend = "lightgbm",
                      covariates = c("ws", "wd", "temp", "RH", "atmos_pres", "blh",
                                     "tcc", "tp", "ssrd", "date_unix", "day_julian",
                                     "weekday", "hour"),
```

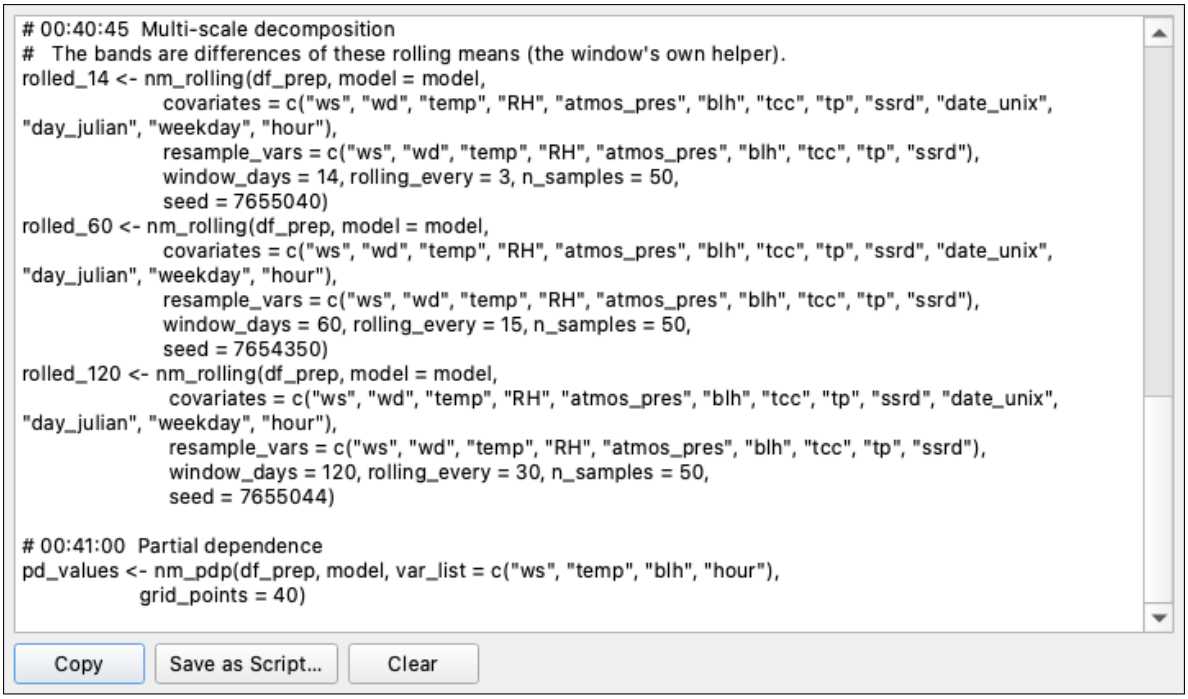


Figure 13.1: The Command Log after this manual's session, its last entries in view: the multi-scale run and the partial dependence, as the calls that repeat them.

```
                    split_method = "random", train_fraction = 0.75,
                    model_config = list(n_trials = 10), seed = 7654321)
df_prep <- res$df_prep
model <- res$model

# 21:34:58 Normalise
normalised <- nm_normalise(df_prep, model,
                        resample_vars = c("ws", "wd", "temp", "RH", "atmos_pres",
                                          "blh", "tcc", "tp", "ssrd"),
                        n_samples = 300, seed = 7654321,
                        return_quantiles = c(0.05, 0.25, 0.75, 0.95))
fig_normalise <- nm_plot_normalise(normalised, ci_low = "q050", ci_high = "q950")
if (interactive()) print(fig_normalise)

# 21:46:18 Decompose
decomposition <- nm_decompose("meteorology", df = df_prep, model = model,
                           target = "value",
                           covariates = c(...the same 13...),
                           n_samples = 100, seed = 7654321,
                           groups = list(Wind = c("ws", "wd"),
                                         `Temperature and humidity` = c("temp", "RH"),
                                         `Boundary layer and pressure` = c("atmos_pres
                                             ", "blh"),
                                         `Sky and rain` = c("tcc", "tp", "ssrd")))
fig_decomposition <- nm_plot_decomposition_stack(decomposition)
if (interactive()) print(fig_decomposition)

# 21:47:17 Rolling normalisation
rolled <- nm_rolling(df_prep, model = model, covariates = c(...),
                     resample_vars = c(...the nine...),
                     window_days = 14, rolling_every = 7, n_samples = 50,
```

```
                    seed = 7654321)
fig_rolled <- nm_plot_rolling(rolled, data = df_prep)
if (interactive()) print(fig_rolled)
```

Run again with `Rscript`, or `source()` in R, it gives the same model and the same results: every call carries its seed, and the LightGBM search runs a fixed number of trials rather than for a time.

13.3 Figures for publication

A figure on screen is drawn at the tab's size; shrunk into a journal's 89 mm column its lettering shrinks with it. *For print…*, on every figure's toolbar, lays the figure out again at the size the journal prints it, each text at the size its guide gives that kind of text — a title, an axis label, a tick label, a legend — with the margins and the ticks worked out anew for that size, not scaled to it (Figure 13.2):

Journal Elsevier, Nature, Copernicus (ACP, AMT, …) or ACS (ES&T), each with what its own artwork guide asks, and a link to that guide.

Width The journal's columns (Nature 89 or 183 mm; Elsevier 90, 140 or 190 mm; ACS 3.33 or 7 in; Copernicus at least 80 mm), or a width of your own. A figure of one column of panels starts at the narrowest, panels side by side at the widest.

Height As the figure needs — its own proportion, raised until every panel has at least 12 mm for its data, within the journal's greatest height, and lowered to what is drawn where panels of a shape of their own (roses side by side) would leave the rest blank — or set.

Panel labels None, a b c, (a) (b) (c), A B C or (A) (B) (C), bold or not, from any letter (a figure that is panels c and d of a composite starts at c); each column of panels has its letters in a line.

Line width, Marker size The data lines' stroke; a time series takes round markers when the size is raised. Reference lines keep theirs.

Legend Where it sits (or drag it), its text size, its columns, its frame. A legend laid out in a row wider than the printed figure is given fewer columns.

Formulas Written as chemistry writes them: NO_2, $PM_{2.5}$, SO_4^{2-}.

Titles The figure's and the panels' titles are left to the caption, as journals ask; ticked, they are drawn.

Raster resolution 300, 600 or 1200 dpi, for TIFF and PNG.

Under the settings, the checks read the figure as it is laid out against the journal's guide: its width and height, its smallest text and its smallest sub- or superscript (Nature asks for 5 pt at least and 7 pt at most, Elsevier 7 pt, ACS 8 pt), the face it is drawn in (Arial or Helvetica where the guide names them), tick labels that run into one another, and how many panels it has (ACS: four at most); a check the figure fails is shown in red, and all are read again as the settings change. The preview fits the window, or shows the figure at its actual printed size. On the preview, double-click a title, an axis label or a panel letter to edit its text and size, a tick's numbers to size and turn them (and give that tick a text of its own), the legend to place it, or a colour bar to put it on another side of its panel and give its width and height on the page in millimetres; drag the legend or a colour bar where it reads best. *Save…* writes the formats the journal takes — PDF, EPS or SVG with the text kept as editable text and the fonts embedded, TIFF or PNG at the resolution chosen — at exactly the size shown, on a white page. While the dialog is open the figure is in the report's language (the one last picked under File → Export HTML Report). When it closes the figure goes back to its tab as it was; the journal, width, panel labels, formulas and resolution are kept for the next figure. Check the journal's current guide before submitting.

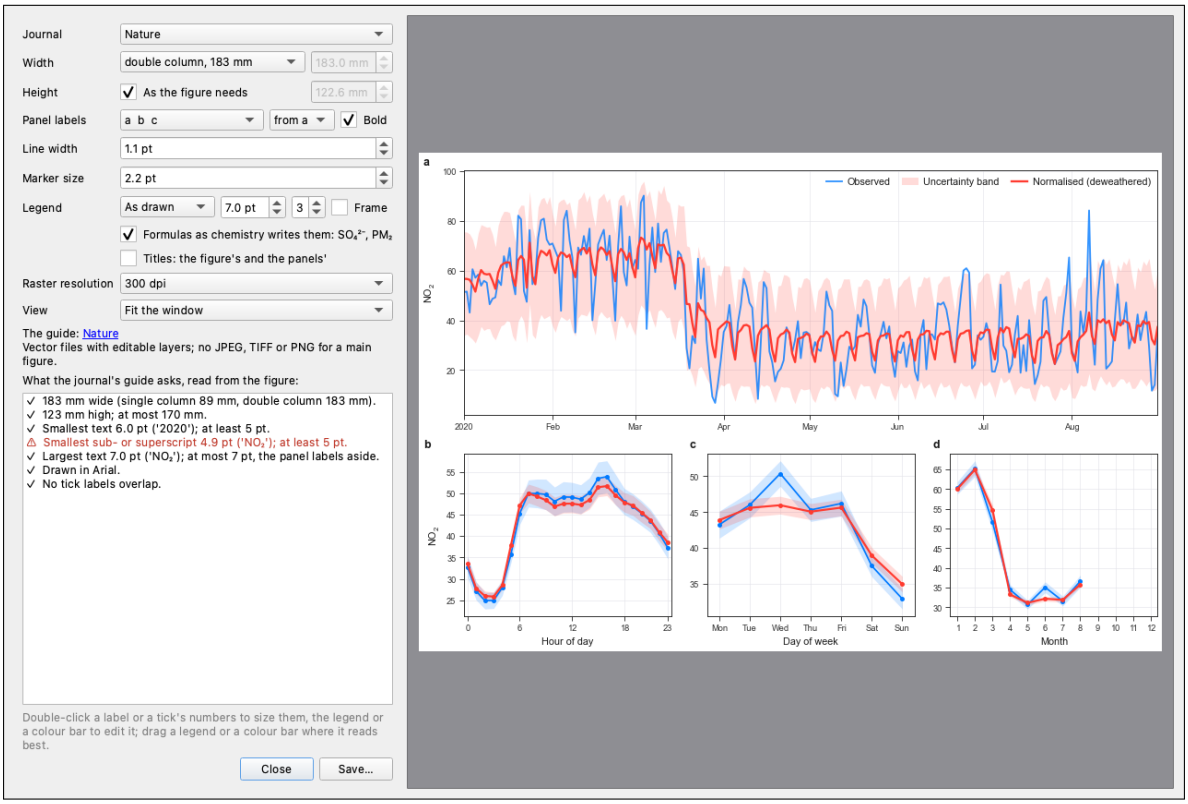


Figure 13.2: The normalised series laid out for a Nature double column: 183 mm wide, its panels lettered, beside what Nature's guide asks of it — a subscript at 4.9 pt, below the 5 pt Nature asks, is the one check it fails: Nature's 7 pt ceiling leaves no room for more.

13.4 Saving and reopening a session

File → Save Results... (Ctrl+S) writes the session to one file (`.normet`): the table, the panel's settings, the model, every step's result and the command log. **Load Results...** reads one back into the window as it was — each result is handed to its step's own handler, so its tab, verdict and figure are drawn again — and a notice says what was restored. After each step that finishes, the window also keeps the session as `~/.normet/last_session.normet`; **Reopen Last Session** reads it, should the window have been closed, or have crashed, before the results were saved. A session file is a Python pickle, with the R models' folders inside it, so it opens on another machine: open only one you or the window wrote.

File → Load Session Script... runs a script the command log wrote (Section 13.2) and brings its results into the window the same way: its table, model and results, and the panel set from the calls it made — read by R's own parser, before it runs. The script is R and runs as it is written, in R, from the folder it was saved in (so a table it reads by a relative path is found); the window asks first.

13.5 A Methods paragraph

File → Methods Paragraph... writes what the session ran as a paper's Methods says it, ready to copy (Figure 13.3): the record (its length, span and step), the model (backend, search, predictors, split and seed) and its held-out fit, then each step run — the normalisation's draws and resampled variables, the decomposition, the rolling windows, the multi-scale widths, the partial-dependence variables — with the references the methods rest on. It is written from each step's call as it ran, not from the panel as it stands, and in the window's language. Check it against what you did.

13.6 Configurations and history

File → Save Config... writes every setting of the panel to JSON — target, predictors, backend, split, budget, estimators, seed, and every step's settings — and **Load Config...** sets the panel from one; the data are not part of it. **File → Run History...** lists every run with its time, task, target, backend, rows and a detail (the held-out R^2 for a training run, the samples for a normalisation).

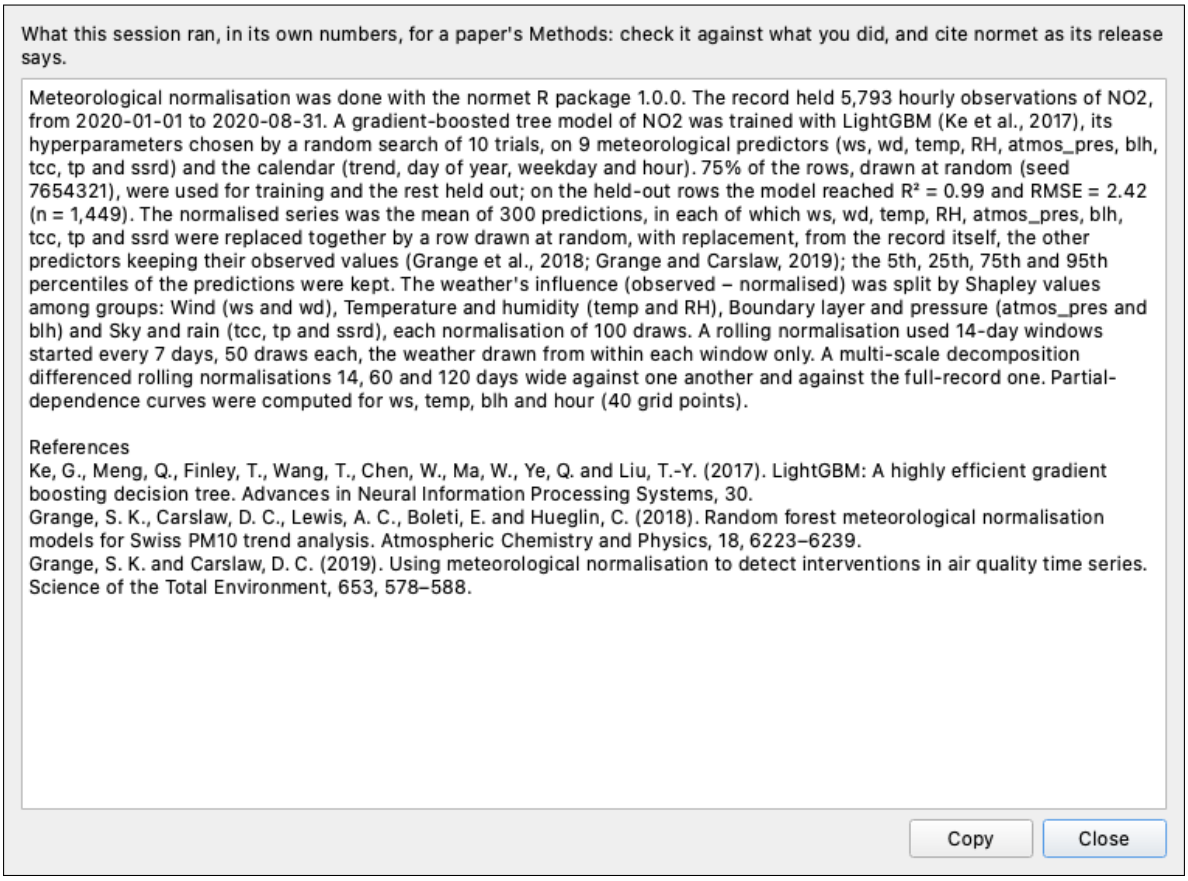


Figure 13.3: The Methods paragraph of this manual's session: the record, the LightGBM model and its held-out fit, and each step that was run, with its references.

Chapter 14

The Assistant

The Assistant is a language model working in the window. It is open on the right when normet starts; **View** → **Assistant** (Ctrl+Shift+A) closes it and opens it again. It reads what the window holds — the table, the model's held-out fit, the normalised series, the decomposition — as normet's own readings of them, in text, and runs the window's steps the way their buttons do, so what it does appears on screen as your own runs do and goes into the command log.

The conversation reads as a chat does: your messages on the right, the replies as their Markdown reads (code shaded, tables ruled), each tool the Assistant calls one quiet line — its dot grey while it runs, green when it answered, red when it was refused, the refusal under it — whose result opens with a click. Ctrl/Cmd + and − (or the wheel with the key held) size its text, and Ctrl/Cmd 0 resets it; the size is kept. You write in the rounded box at its foot, which grows with what you write: Enter sends, Shift+Enter starts a new line. *Send*, *Stop* and *New chat*, which starts the conversation again, are inside the box.

Where it helps most is what to model and how to read the result. It *proposes* a target and predictors as a card with an *Apply* button, saying what it left out and why; nothing in the panel changes until you click Apply, and it trains or normalises only once you have agreed. It is told to quote only numbers the tools return.

14.1 Full automatic

Auto, beside *Key...*, lets the Assistant work without waiting for you. With it on, the target and predictors it proposes are applied at once (the card says *Applied*), and it works the analysis through in the window's order — the data, the set-up, training, normalisation, decomposition — reading each result as it comes. A model that stops with steps left is told what is left and goes on, up to eight times; two reminders in a row that change nothing end the turn, as a model that cannot do a step should not circle. A call to the model that fails, or an empty reply, is made again, twice at most. Each reminder and retry is a line in the conversation. *Stop* ends it at any time; the choice is kept for the next session, and *What full automatic does...* under the button's arrow says all this. A command-line agent has the set-up applied at once too, and works the chain as far as its own loop allows.

14.2 Choosing a model

An API model named `provider:model`: `anthropic`, `openai`, `deepseek`, `gemini`, or a local `ollama` or `lmstudio` server — for example `anthropic:claude-sonnet-5-5`, `deepseek:deepseek-chat`, `ollama:qwen3:8b`. Its key is read from the macOS keychain (*Key...* stores it there) or the provider's environment variable (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `DEEPSEEK_API_KEY`, `GEMINI_API_KEY`), and written nowhere else. Local servers need no key.

A command-line agent signed in to your own plan: Claude Code, Codex, Google Antigravity's `agy` or GitHub Copilot, with the effort it thinks at. The window serves its tools to the agent over MCP on 127.0.0.1 with a token drawn for that window, and gives it no shell and no files of its own.

What the Assistant sends to a remote model is the readings — summaries, statistics, captions — and your messages, not the table.

Chapter 15

Scripting: R and MCP

15.1 The R package

Everything the window does is a function of the normet R package. The one-call pipeline prepares, trains and normalises:

```
library(normet)

met <- c("ws", "wd", "temp", "RH", "atmos_pres", "blh", "tcc", "tp", "ssrd")
predictors <- c(met, "date_unix", "day_julian", "weekday", "hour")

res <- nm_do_all(my1, target = "NO2", backend = "lightgbm",
                 covariates = predictors, resample_vars = met,
                 n_samples = 300, n_cores = 4)
nm_modStats(res$df_prep, res$model) # training, testing and all rows
```

Step by step:

```
built <- nm_build_model(my1, target = "NO2", backend = "lightgbm",
                        covariates = predictors, model_config = list(n_trials = 20))
df_prep <- built$df_prep
model <- built$model
norm <- nm_normalise(df_prep, model, resample_vars = met, n_samples = 300,
                     return_quantiles = c(0.025, 0.975))
auto <- nm_normalise_auto(df_prep, model, resample_vars = met) # stops when converged
emi <- nm_decompose("emission", df = df_prep, model = model, covariates = predictors)
mets <- nm_decompose("meteorology", df = df_prep, model = model,
                     covariates = predictors,
                     groups = list(wind = c("ws", "wd"), rest = met[-(1:2)]))
roll <- nm_rolling(df_prep, model, covariates = predictors, resample_vars = met)
curves <- nm_pdp(df_prep, model, var_list = c("ws", "temp"))
```

Further functions, all exported:

Function	Does
`nm_do_all_unc`	the pipeline over `n_models` seeds, with confidence bands
`nm_do_all_multisite`	per-site pipelines
`nm_cv_score`	walk-forward cross-validation
`nm_run_scm`, `nm_scm_all`	a synthetic control; every unit as if treated
`nm_placebo_in_space`, `nm_placebo_in_time`	placebo tests
`nm_uncertainty_bands`, `nm_effect_bands_space`	jackknife/bootstrap and placebo bands
`nm_scm_diagnostics`, `nm_conformal_effect_interval`	fit diagnostics; conformal interval
`nm_bayesian_scm`	Bayesian SCM
`nm_detect_events`, `nm_anomaly_scores`	events in a normalised series
`nm_plot_normalise`, `nm_plot_decomposition_stack`, `nm_plot_scm_dashboard`	figures (ggplot2)
`nm_make_run`, `nm_save_run`, `nm_generate_html`	provenance and reports
`nm_make_cache`	on-disk caching

Each function's help page (`?nm_normalise`) gives its arguments and an example, and the reference at https://github.com/apai-sys/normet-r lists them by area.

15.2 Rscript

The package has no command of its own: an R script runs with `Rscript`, and the script the command log saves is one (Section 13.2):

```
Rscript normet_session.R # repeats the window's session
```

`nm_make_run` and `nm_save_run` keep a result with its provenance (an `.rds` with a `.meta.json` beside it), which `nm_generate_html` turns into an HTML report.

15.3 AI assistants over MCP

`nm_mcp_server()` serves normet to an AI assistant over the Model Context Protocol, on stdio:

```
claude mcp add normet -- Rscript -e "normet::nm_mcp_server()" # Claude Code
# Claude Desktop, claude_desktop_config.json:
# {"mcpServers": {"normet": {"command": "Rscript",
# "args": ["-e", "normet::nm_mcp_server()"]}}}
```

Its tools are normet-py's `normet-mcp`'s, by the same names: the assistant can list the examples, inspect a table, start do-all, decompose, SCM and cross-validation jobs, follow and cancel them, read a run or a result, draw a figure and write a report. Each job is an R script, `job.R`, run by `Rscript` in a folder of its own (`$NORMET_MCP_HOME`, `~/.normet/mcp-r` by default), so the script is its record and runs again without the assistant. The assistant is given normet's readings of a table or result, not the data; files are read only under `$NORMET_MCP_ROOTS` (your home folder by default). The server asks the assistant to propose the target, predictors and donor pool with its reasons, to let you confirm them, and to quote only the numbers the tools return. In R, `nm_agent_tool()` calls the same tools.

Appendix A

Defaults

Setting	Window	Library
Split method / training fraction	random / 0.75	random / 0.75
LightGBM search trials	20	50, each scored by 5-fold CV
H2O time budget	60 s	60 s
H2O algorithms	GBM	GBM
Seed	7654321	7654321
Normalisation samples	300	300
Resampled variables	recognised meteorology	every non-calendar predictor
Quantiles	5, 25, 75, 95 % when ticked	none
Decomposition samples	300	300
Attribution	auto	sequential (Shapley with groups)
Rolling window / step / samples	14 d / 7 d / 100	14 d / 7 d / 300
Multi-scale widths	14 / 90 / 365 d	—
PDP grid points / range	50 / 1st–99th percentile	50 / 1st–99th percentile
`nm_normalise_auto`	—	batches of 100, P95 RSE $< 3\,\%$, 3 in a row, ≤ 5000
SCM backend	scm	scm
Placebo in time: min pre / every	30 / 7 steps	—
Uncertainty bands	jackknife, $B = 100$	jackknife, $B = 200$, 80 % donors

Appendix B

Where normet keeps its files

What	Where
Recent tables (eight)	`~/.normet/gui_recent_files.json`
Run history	`~/.normet/gui_run_history.jsonl`
Window geometry, appearance, language, Assistant model, Rscript	the system's settings store (`apai-sys/normet-r-g`
Font cache (installers)	`~/.normet/matplotlib`
The last session (after each step)	`~/.normet/last_session.normet`
Assistant API keys	macOS keychain, service `normet`
MCP jobs	`$NORMET_MCP_HOME`, default `~/.normet/mcp-r`
Each call into R, its tables and models	a temporary folder, removed when the window clos

Nothing else is written unless you save it. The print dialog's choices, the Assistant's text size and the report's language are kept in the settings store with the rest.

Appendix C

Troubleshooting

"The CSV must contain a 'date' column" Rename the timestamp column to `date`. Panel data (several units) belongs in the SCM Studio.

The model's verdict is red The predictors do not explain the target. Check that the meteorology is ticked and on the same time base as the target, raise the search trials, or try H2O. Normalising with a poor model returns little more than the observed series.

A meteorological column is not ticked Only the names in Section 6.1 are recognised; tick others by hand, on the **Data** and **Normalise** pages.

Exact Shapley values over k features need 2^k normalisations More than ten variables by Shapley: give groups, or use sequential attribution.

"... not among the variables trained on" A group (from a configuration read in) names a variable the model was not trained on; the message names it.

"R was not found" Install R, or point the window at `Rscript` with **R** → **Locate Rscript...**.

"the normet package could not be loaded" Install it in R: `remotes::install_github("apai-sys/normet-r")`; then **R** → **Check R environment**.

"Neither lightgbm nor h2o loads in R" `install.packages("lightgbm")`; H2O also needs Java.

Multi-scale asks for the normalisation first It differences against the normalised series.

A multi-scale band was skipped Its width leaves fewer than two windows on the record; lower it.

SCM: "The pre-treatment fit is no check on this design" A series shuffled out of any relation to the donors fits the pre-period as well as the treated unit does: there are too many donors for the pre-cutoff dates. Judge the effect by Placebo in space and by other estimators, or use fewer, better-chosen donors or a longer pre-period.

SCM: "Donor pool too small" Tick at least two donors.

SCM: no valid fake cutoffs The post-period must fit before the true cutoff with *Min pre-period* steps before it; lower *Min pre-period* or *Placebo every*, or move the cutoff later.

A task runs too long *Cancel* stops it, and the R process running it. Lower the samples, the search trials, or the number of variables decomposed.

A notice asks for a step first It names what is missing and carries the step on a button; it goes by itself once the step has been taken.

The window closed before the results were saved **File → Reopen Last Session** brings back the session as it was after its last step.

The parameter panel is missing It is folded away: *Expand Panel* on the action bar, or Ctrl+B.

A task failed The dialog shows R's error; the full traceback is in the **Log** dock.

macOS: "Normet cannot be opened" The app is unsigned: right-click it and choose *Open* once (Section 2).